

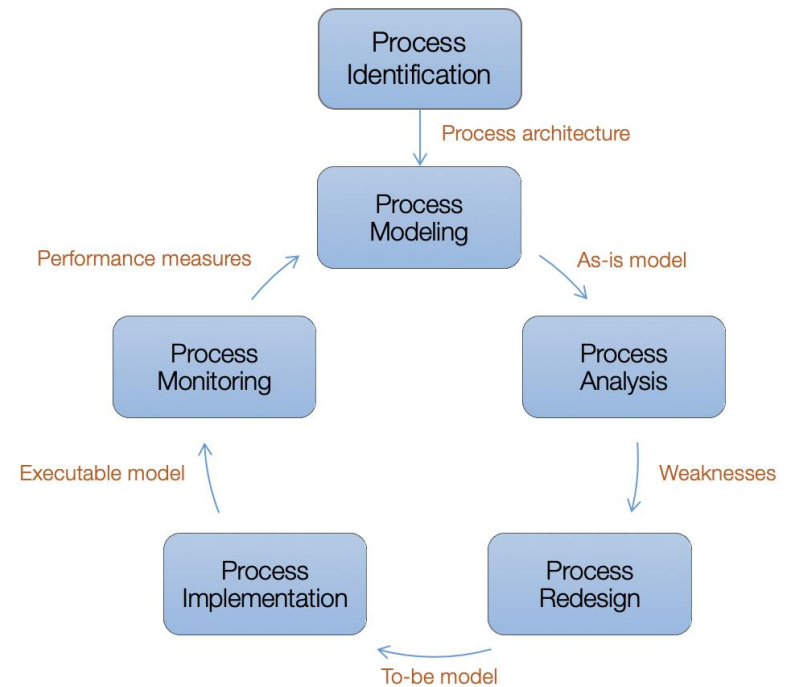


BPMN-BASED BUSINESS PROCESS MODELING & SIMULATION

BPM



- A **Business Process** (BP) is defined as a set of activities that are coordinately performed in an organizational and technical environment to achieve a business goal
- **Business Process Management** (BPM) is referred to as the “*set of methods, techniques and software to design, enact, control and analyze BPs involving humans, organizations, applications, documents and other sources of information*” [van der Aalst et al., 2003]



M&S-based BP analysis



- BP analysis is of paramount importance at:
 - process design/configuration time
 - ✦ to predict its dynamic behavior and avoid costly and time-consuming late reworks
 - process execution time
 - ✦ to identify bottlenecks and define strategies to redesign and improve the process whenever failures or performance downgrades occur
- Effective BP analysis approaches focus on the use of modeling and simulation (M&S) techniques
 - to model a BP and “execute” the model over time in order to assess its behavior, provide answers to what-if questions and/or carry out performance-aware decision making

M&S and BPM



- However, using M&S approaches in the BPM context can be challenging
 - M&S formalisms and tools require a significant know-how and skills business analysts are usually not familiar with;
 - organizations' process data are often disseminated in different information systems and thus building and parameterizing a simulation-based analysis model turns out to be an effort-consuming and error-prone activity
 - large organizations' processes can be highly complex, distributed and multi-disciplinary

Challenge and solution



- Enabling process analysts to cost-effectively carry out simulation-based analysis activities by reducing the effort required to
 - build and execute simulation models of BPs specified by use of *BPMN (Business Process Model & Notation)*
- How?
 - introducing a *model-driven automated framework* that exploits:
 - ✦ *PyBPMN*, a BPMN extension to specify performance-related properties
 - ✦ *eBPMN*, a domain-specific language for business process simulation

BP Modeling – What is a process model?



- A process model is a purposeful abstract representation of a BP, with the intent of capturing specific aspects according to the following properties:
 - Simplification
 - ✦ The model reflects *key aspects* of the process
 - Abstraction
 - ✦ The model captures only relevant aspects for modeling scope *abstracting from* irrelevant ones (principle of *limited substitutability*)
 - Purpose
 - ✦ analyzing existing processes (*descriptive* use)
 - ✦ supporting process design and implementation (*predictive* and/or *prescriptive* use)

BP Modeling – Modeling Languages



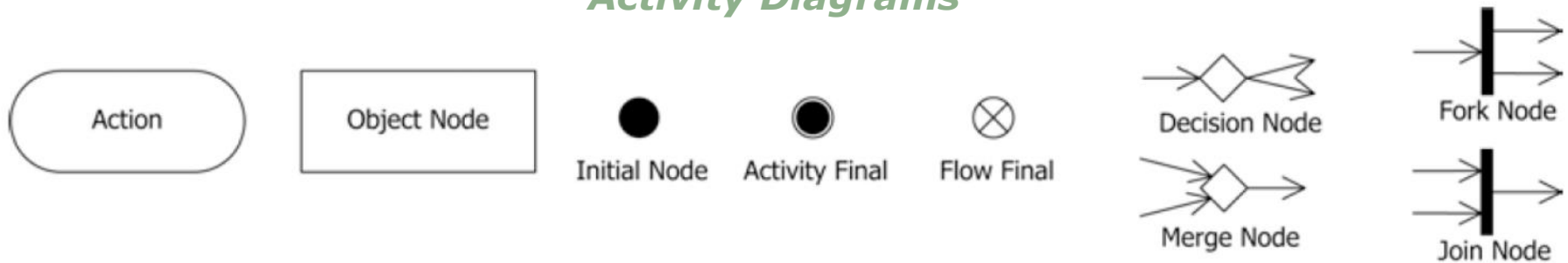
Two main abstraction levels for process modeling can be identified in BPM:

- *Organizational design*
 - *the model is used for a complete understanding of the BP during its lifetime, from communication with stakeholders to BP evaluation and improvement*
 - *the model is intuitive and generally represented with an easy-to-understand graphical notation*
- *Application development*
 - *the model is more detailed and thus includes all the technical information required for the implementation and automation of the BP*

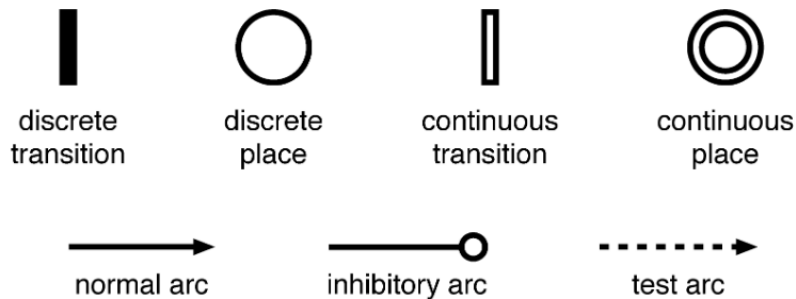
BP Modeling – Languages



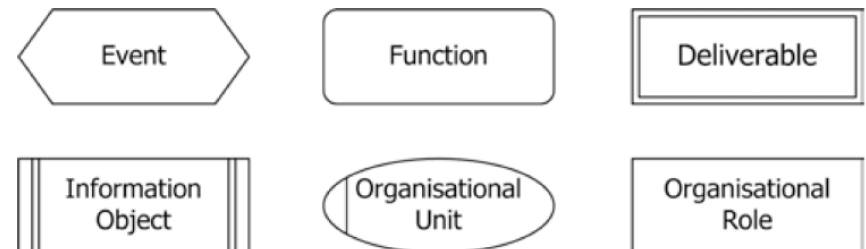
Activity Diagrams



Petri Nets



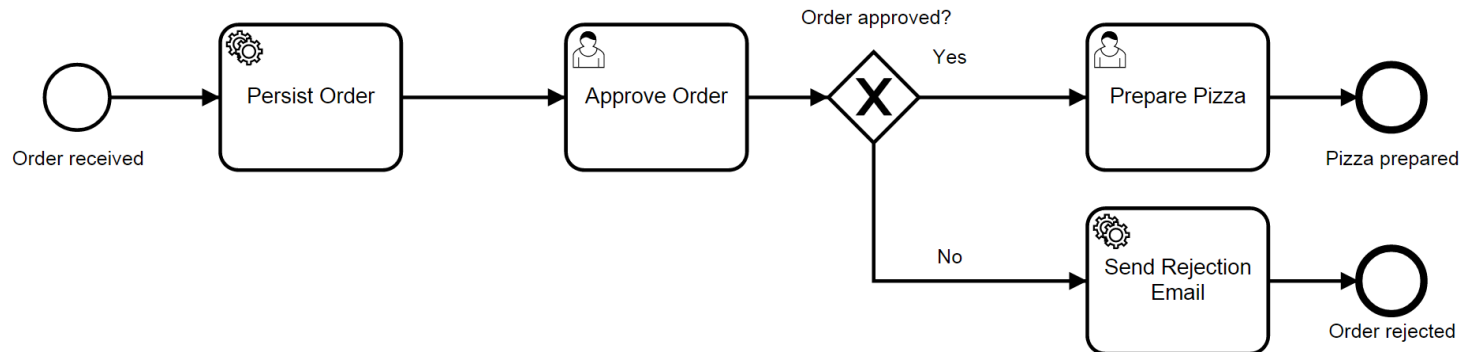
Event Driven Process Chains



BP Modeling – BPMN



- Among all formalisms and languages defined for the specification of BP models, the *Business Process Model and Notation* (**BPMN**) is currently the most used one, being recognized as a *de-facto standard*

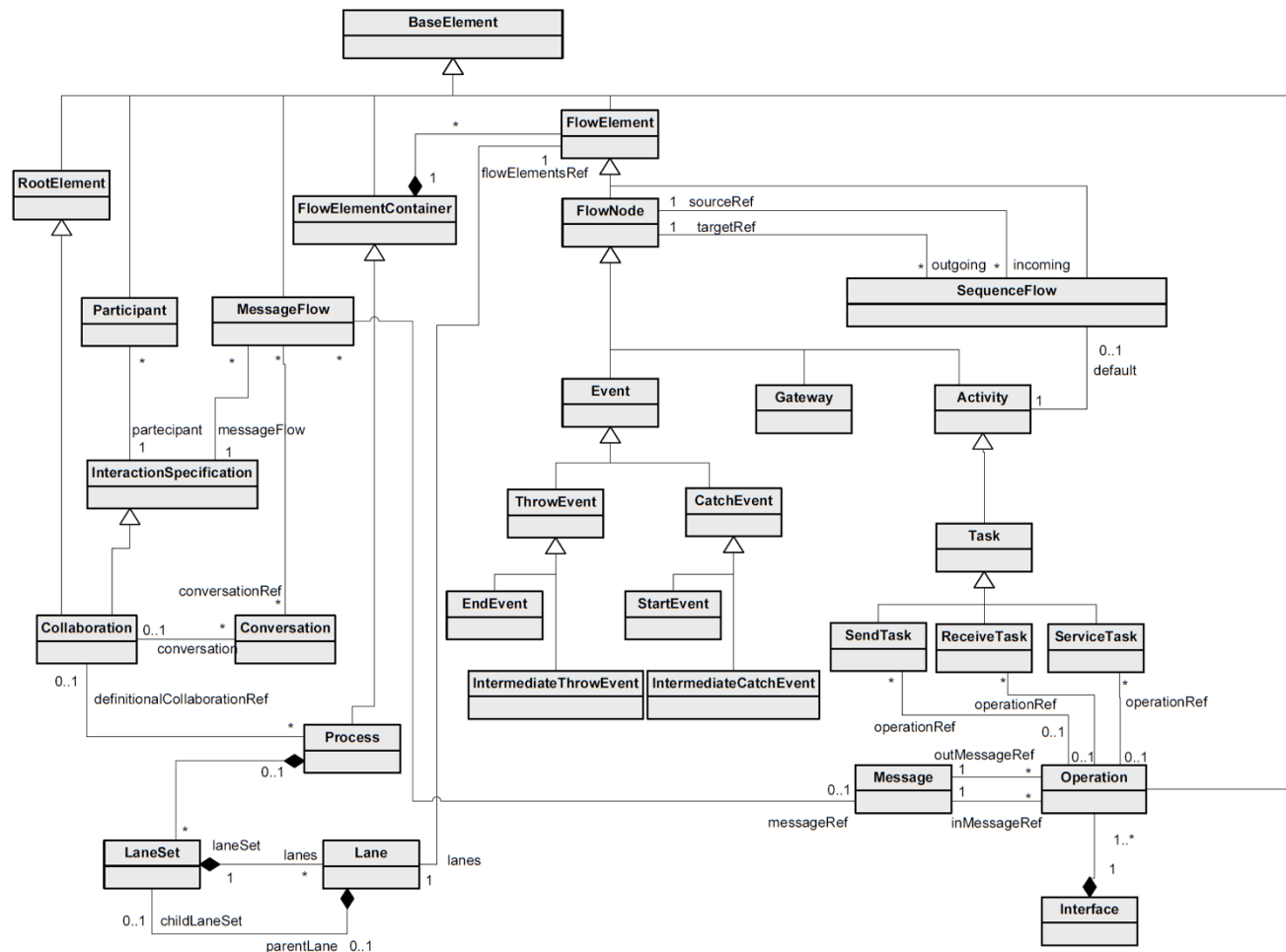


Why BPMN?



- BPMN *execution semantics* is formalized by use of the *token* concept:
 - The “*simulated*” behavior of elements can be defined by describing how they interact with a token
 - The token is not part of the BPMN specification
 - BPMN modeling tools are not required to implement any form of token
 - The token will traverse the sequence flows and pass through the elements in the process
 - ✦ Token traverse sequence flows instantaneously, so *no time concept* is associated to tokens
- BPMN 2.0 has a *formal definition (metamodel)*
 - Precise definition of the constructs and rules for creating models
 - The modeler does not handle the metamodel
 - ✦ The modeling tool ensures that the model conforms to the metamodel

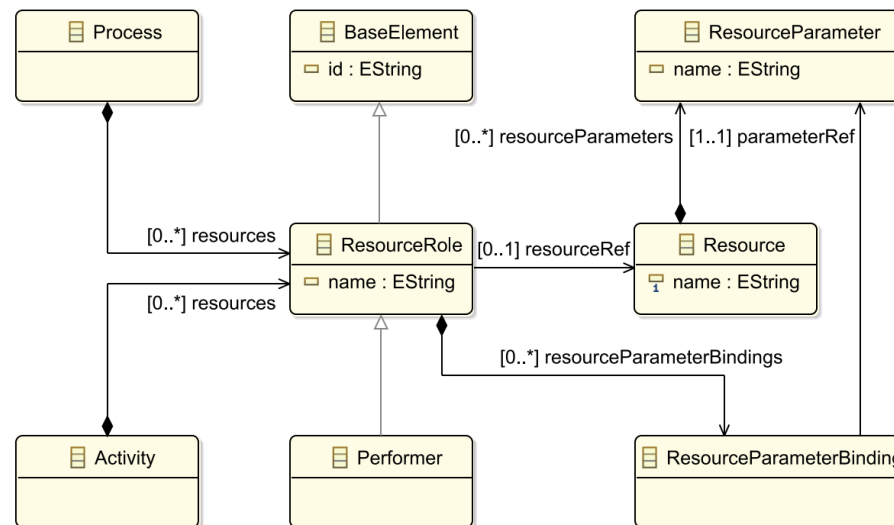
BPM Modeling – BPMN Metamodel



BPMN Metamodel for Resources



- BPMN allows to specify Resources that can be referenced by Process and Activity through the ResourceRole element
- Multiple Activity elements can use the same Resource



BP Simulation



- Considered as the most popular and used technique for quantitative analysis of process models
- A *simulation* is the (computer-based) *execution of a model over time*
 - obtained by implementing a *simulation model* and then running the *model implementation* so to generate and execute process instances, in order to gather data and obtain relevant performance indicators
- The simulation model can be implemented using:
 - GPL
 - Simulation language (specific primitives for the simulation field)
 - Simulation package (tool providing domain-specific building blocks used to produce and execute the simulation model)

BP Simulation – Model Implementation



- **GPL**
 - Highest flexibility
 - Requires a substantial technical know-how and a considerable extra development effort
- **Simulation language**
 - Reduced flexibility
 - Increased cost-effectiveness, but still requires significant technical know-how and effort
- **Simulation package**
 - Lack of flexibility
 - Easier to use, since the package generally provides a GUI for the visual definition of the simulation model

Advantages of BP Simulation



- Business process simulation provides several benefits over other analysis methods:
 - Flexible enough to allow the analysis of complex processes
 - Can be used to analyze several performance indicators at the same time
 - ✦ Waiting times
 - ✦ Resources utilization
 - ✦ Costs
 - ✦ Error rates
 - Can be used to investigate what-if scenarios
 - ✦ Improvement of the BP through the evaluation of various alternatives without carrying out any process implementation activity

Drawbacks of BP Simulation



- Can be time and computational consuming
 - depending on the model resolution and the number of process instances that are executed
- Soundness of simulation results depends on the correctness and completeness of the business process model
- May require software development skills in order to set-up and execute the simulation model

Discrete Event Simulation (DES)



- Most used approach for BP simulation
 - Most business processes can be described as a sequence of separate, discrete, events
- Events cause state changes
- Process state evaluated only at scheduled simulation times
- Scheduling is event-driven
 - Only (simulation) times of events are scheduled

Model Parametrization



- Parameters used to properly *configure* the simulation model, according to the simulation objective
- Parameters also specify *non-functional properties* (NFP) of the BP elements
 - Description of process properties rather than process behavior
- BPMN does not allow business analysts to specify NFPs
 - An additional language or an extension is needed

BPMN Model Parameterization Approaches



- **BPSim**
 - Standard provided by the WfMC and directly linked to BPMN
- **BPMN extension mechanism**
 - Standard extension mechanism introduced with version 2.0 of BPMN
- **BPMN metamodel extension**
 - Lightweight metamodel extension through standard MOF-based profiling mechanism
 - ✦ PyBPMN

BPSim



- Standard specification from the *Workflow Management Coalition (WfMC)*
- Allows the business process models, expressed in BPMN or XPD, to be enriched with information for the model analysis
- BPSim specification defines the parameterization of process data for the analysis of process models
- BPSim data can be extended with proprietary vendor information through an extension capability of the metamodel

BPSim Limitations



- BPSim extends BPMN to specify the quantity and the availability of resources
- However, BPSim defines the availability of a resource as a boolean value
 - Resource operable or not according to a calendar parameter, thus requiring to plan in advance its availability
- This semantics does not consider unforeseen events that can implicate the unexpected failure of a resource, i.e., the degree to which an element is operable

BPMN Extension Mechanism



- BPMN metamodel provides an extension mechanism
 - the extended metamodel is still BPMN-compliant
- The extension mechanism is used to “attach” additional attributes and elements to standard and existing BPMN elements
- The adopted approach is referred to as *extension by addition*
 - Standard elements are still intact and can still be understood by other BPMN adopters
 - Different from *extension by specialization* used by UML profiles

BPMN Extension Mechanism Pitfalls



- Lack of methodological support for the creation of BPMN extensions
 - a method for the straightforward development of extensions is missing
- No graphical notation for the representation of BPMN standard extensions
 - Designers are exposed to low level XML-based implementation issues

BPMN Metamodel Extension



- The extension by introducing new concepts into the original BPMN metamodel (i.e., extension by specialization) can be considered equivalent to the extension by addition
 - The former can be easily transformed into the latter using a metamodel-based automated approach
- *Extension by specialization* provides more flexibility
 - Not locked to any specific version of the BPMN standard

PyBPMN

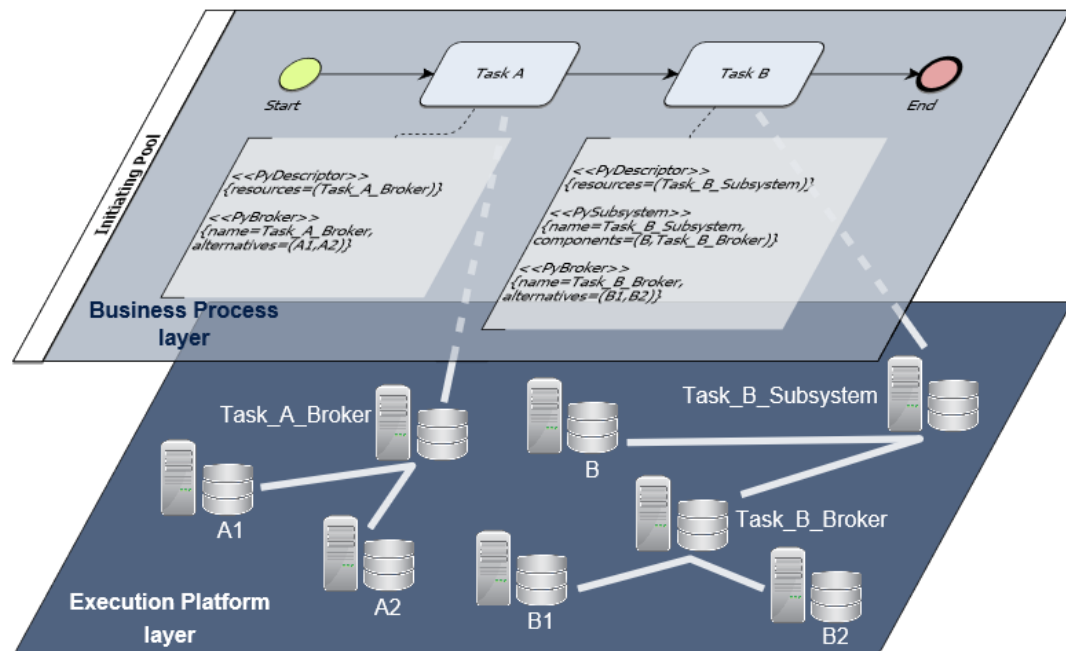


- PyBPMN (Performability-enabled BPMN) is a lightweight BPMN extension that addresses the specification of non-functional properties of business processes, in terms of:
 - Workload
 - Performance
 - Reliability
 - Resource management

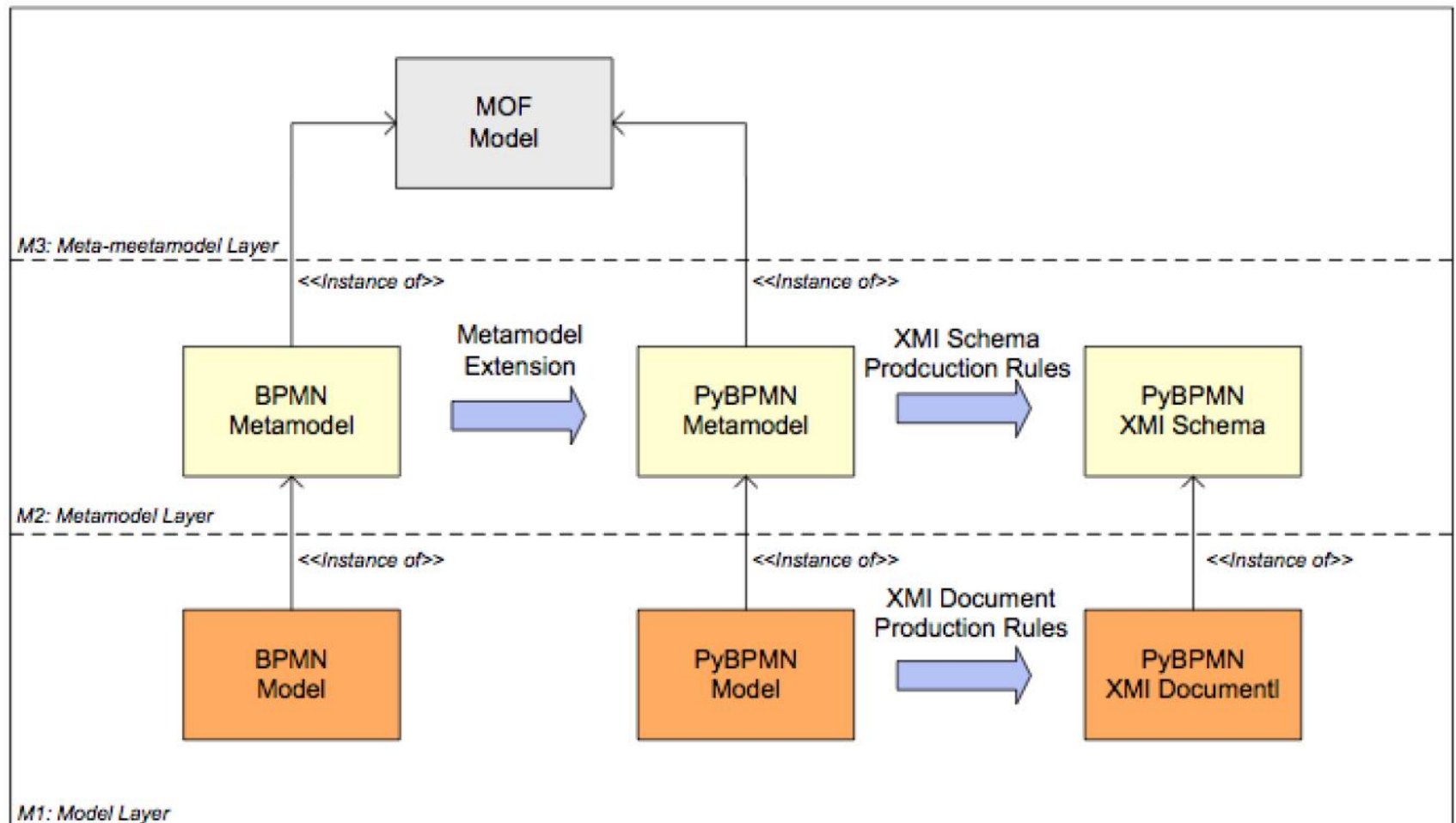
PyBPMN Execution Platform



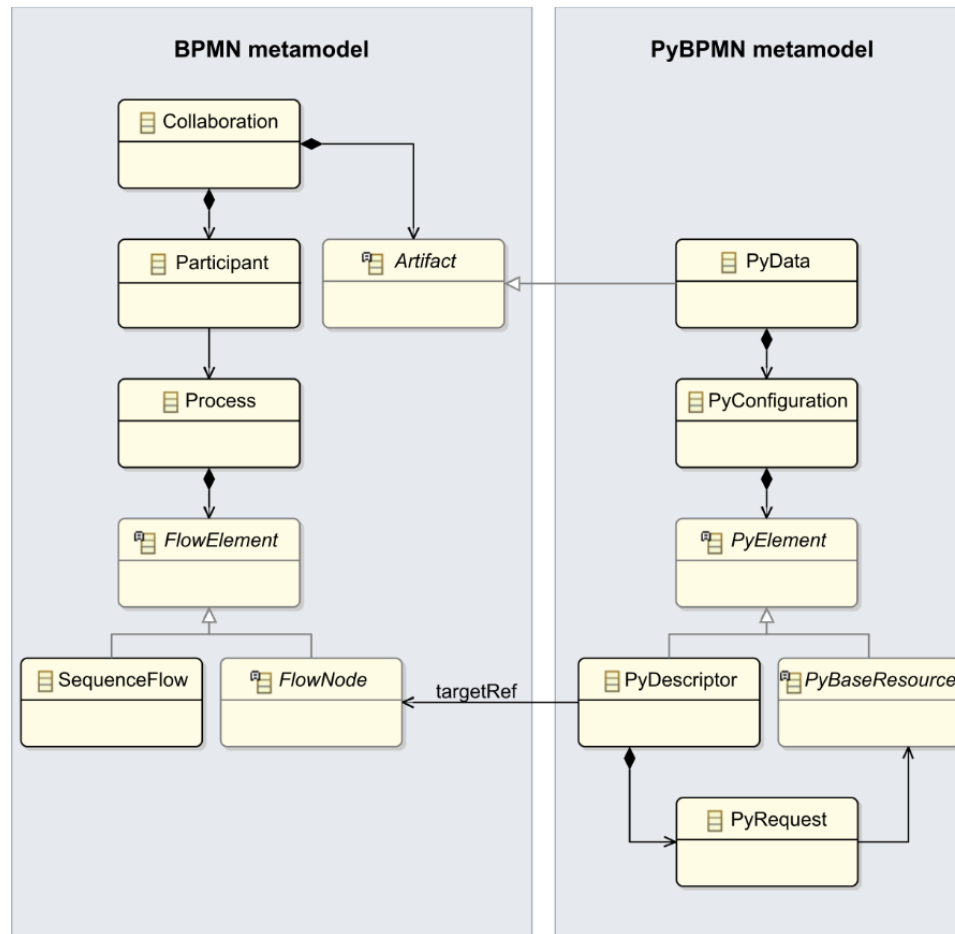
- Provides an execution layer underlying the business process layer
- PyBPMN models elements in the underlying layer refer to BPMN elements in the upper layer



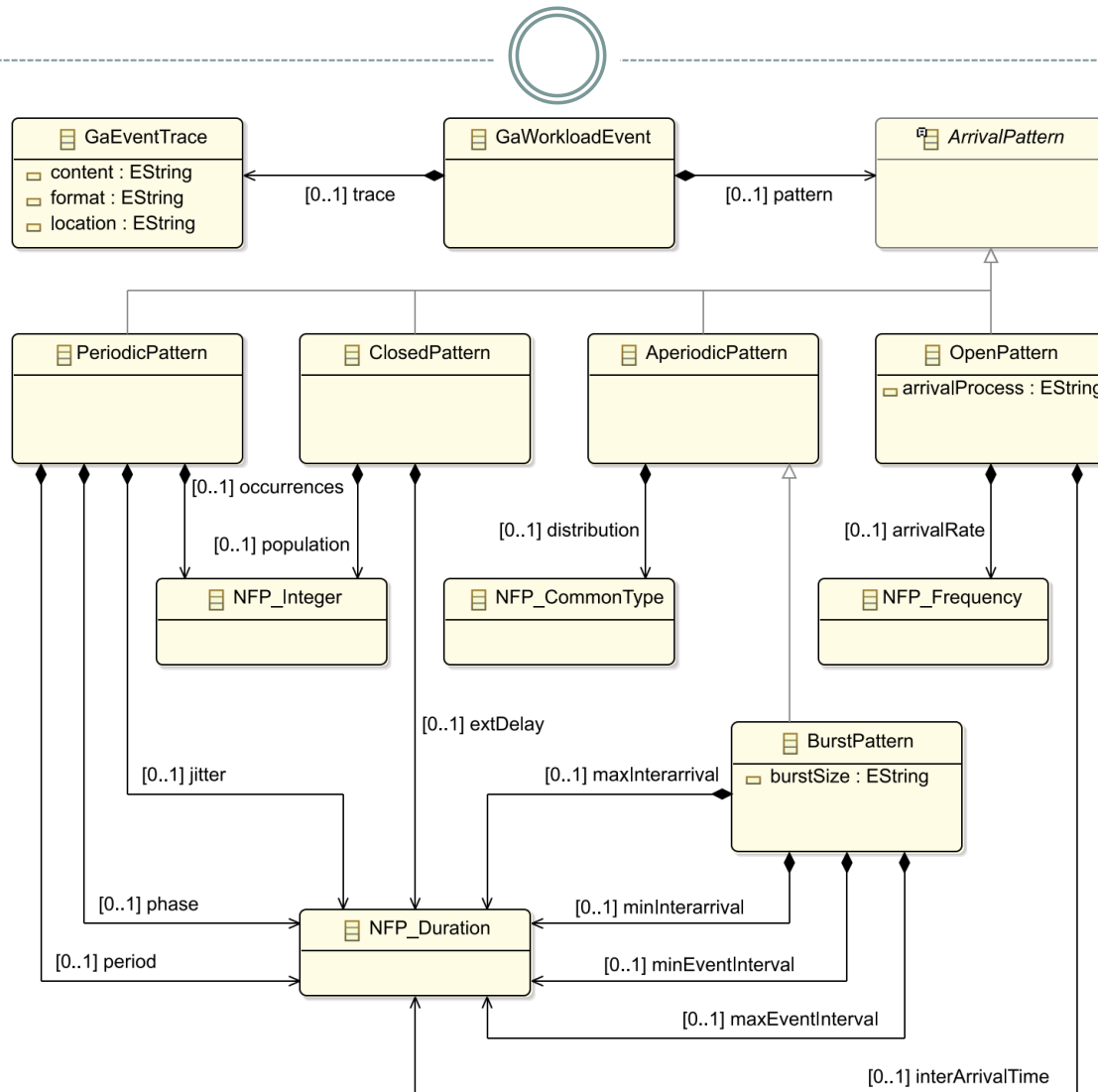
PyBPMN: a metamodel extension



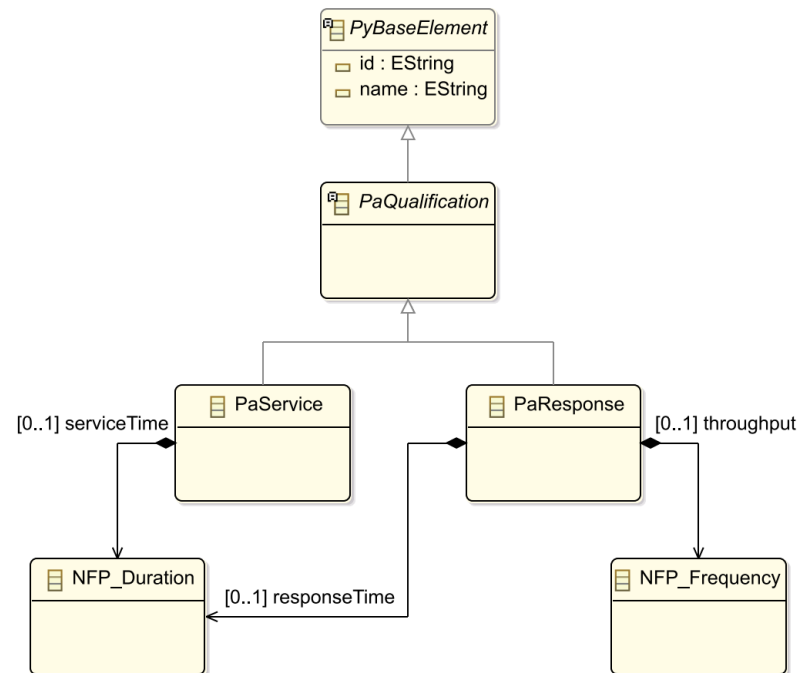
PyBPMN MM Overview



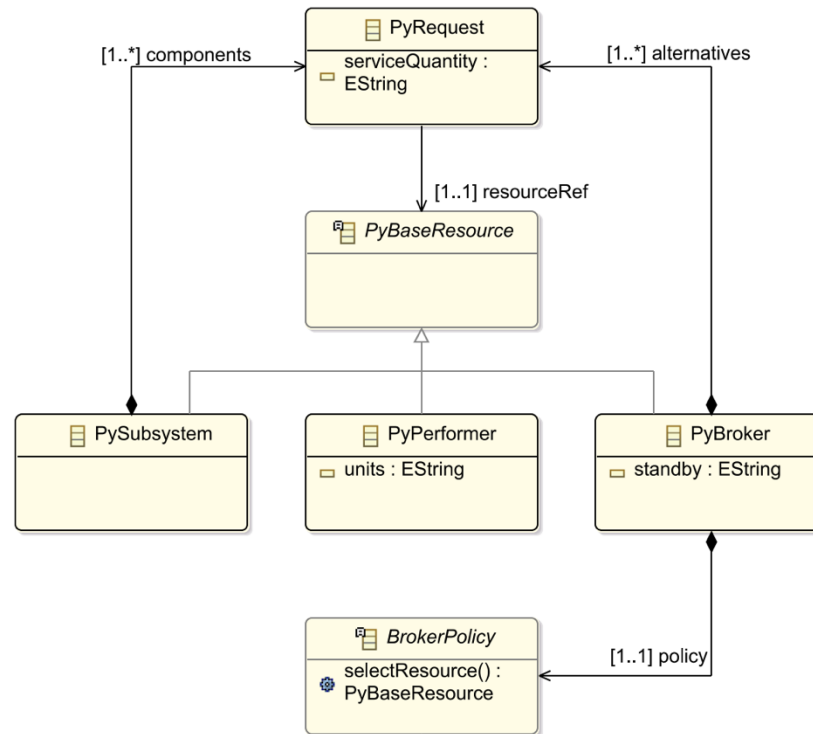
PyBPMN MM Workload



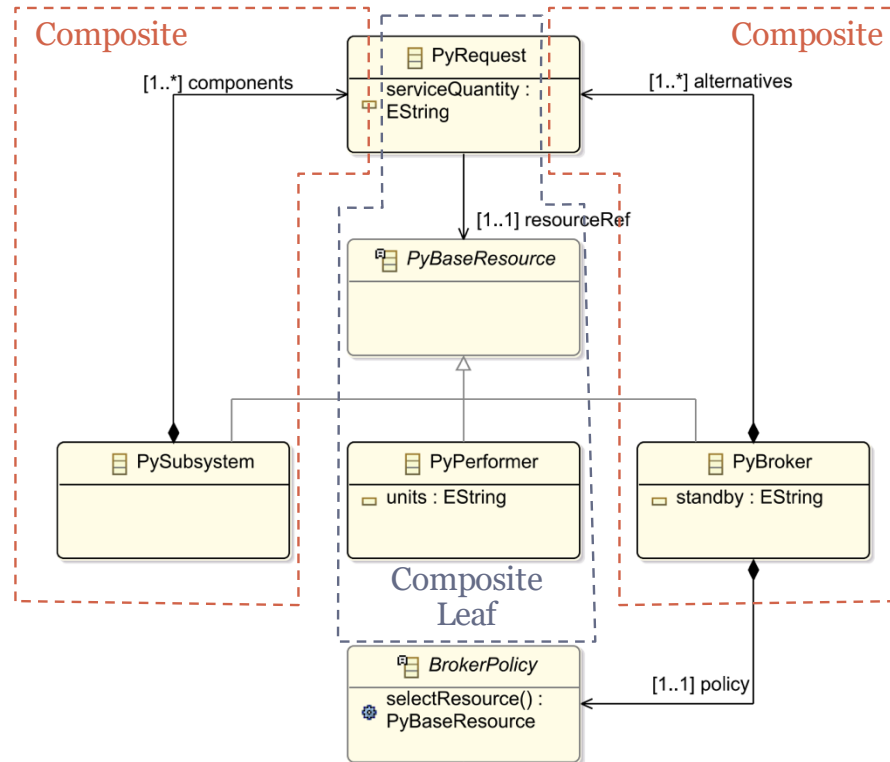
PyBPMN MM Performance



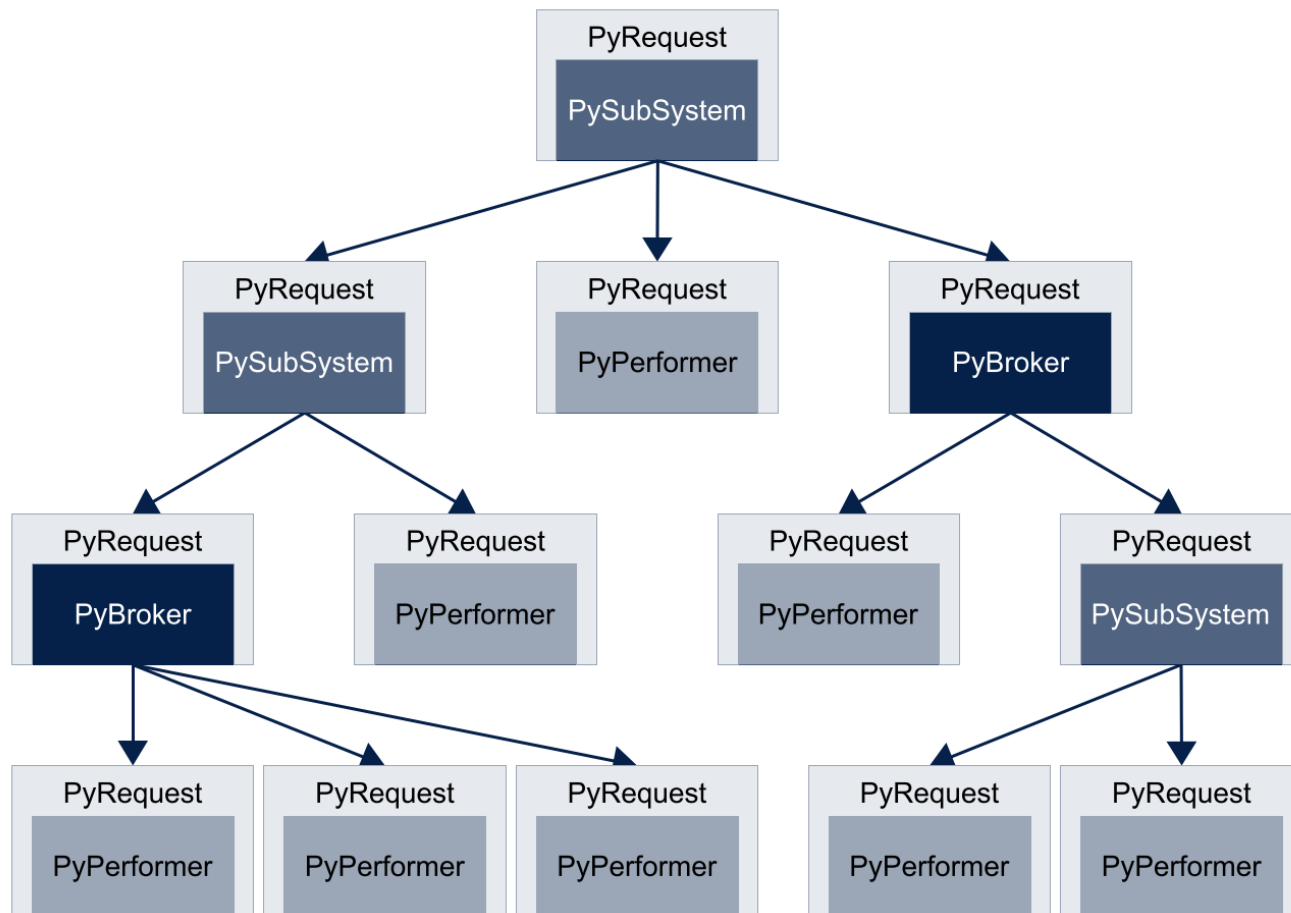
PyBPMN MM Resources



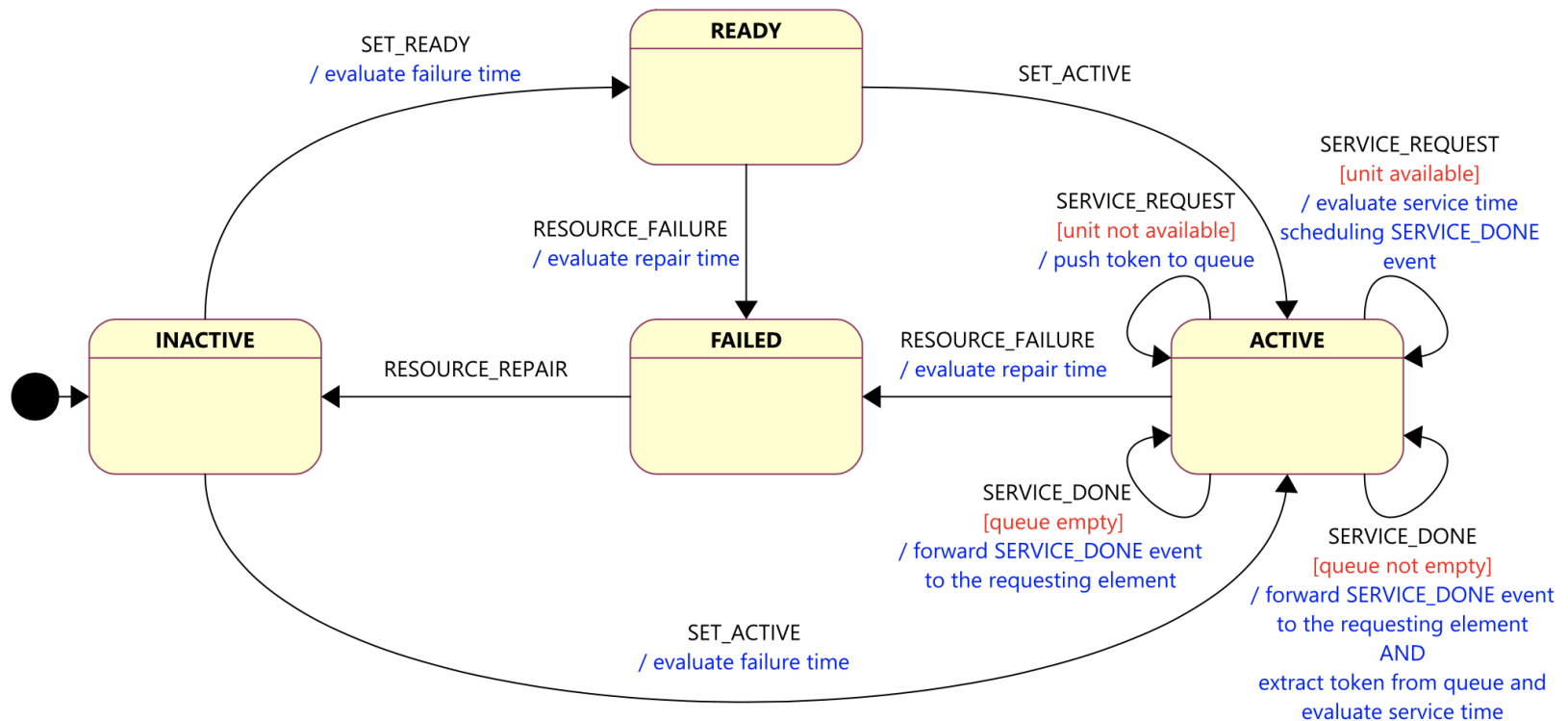
PyBPMN MM Resources



PyBPMN Complex Resources



PyBPMN Resource Lifecycle



BP Model Annotation



- The original BPMN model is annotated by using `TextAnnotation` elements having a specific syntax

$\langle \text{PyDefinition} \rangle ::= \langle \text{PyElement} \rangle \text{ '}' \langle \text{PyParamList} \rangle \text{ '}'$

$\langle \text{PyElement} \rangle ::= \text{ '<<PyDescriptor>>' }$
| $\text{ '<<PyPerformer>>' }$
| '<<PyBroker>>'
| $\text{ '<<PySubsystem>>' }$

$\langle \text{PyParamList} \rangle ::= \langle \text{String} \rangle \text{ '=' } \langle \text{Value} \rangle (\text{ ',' } \langle \text{String} \rangle \text{ '=' } \langle \text{Value} \rangle)^*$

$\langle \text{Value} \rangle ::= \langle \text{Literal} \rangle \mid \langle \text{ComplexValue} \rangle \mid \langle \text{CollectionValue} \rangle$

$\langle \text{Literal} \rangle ::= \langle \text{String} \rangle \mid \langle \text{Number} \rangle$

$\langle \text{ComplexValue} \rangle ::= \text{ '(' } \langle \text{String} \rangle \text{ '=' } \langle \text{Literal} \rangle (\text{ ',' } \langle \text{String} \rangle \text{ '=' } \langle \text{Literal} \rangle)^* \text{ ')' }$

$\langle \text{CollectionValue} \rangle ::= \text{ '(' } \langle \text{Literal} \rangle (\text{ ',' } \langle \text{Literal} \rangle)^* \text{ ')' }$

BP Model Annotation - Definition of Resources



- PyPerformer for actual resources
- PyBroker and PySubsystem for virtual resources composed of other resources (actual or virtual)

```
<<PyPerformer>> {  
    name = Printer,  
    serviceTime = (value=1, unit=s)  
}
```

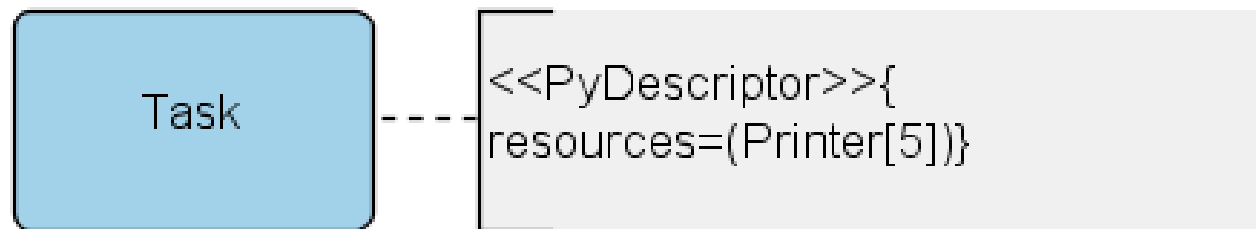
```
<<PyBroker>> {  
    alternatives=(Printer1, Printer2)  
}
```

```
<<PySubsystem>> {  
    components=(Printer,  
    NetworkAdapter)  
}
```

BP Model Annotation - Resources Association



- Which resources are used by a task
- `PyDescriptor` with a comma separated the list of resources used (optionally with service demand)
 - e.g., a printer has a service time of 1 second per page and the task requires printing 5 pages



Model Implementation and Execution



- The simulation model implementation often introduces a significant semantic gap between the BP model and the simulation model
 - when using approaches based on GPL or simulation languages
- Model transformations contribute to reduce the gap
 - But still require the specification of a valid model transformation
- We have introduced a domain-specific simulation language to facilitate the definition of model transformations from BP models to simulation model implementations
 - The *eBPMN* language

eBPMN



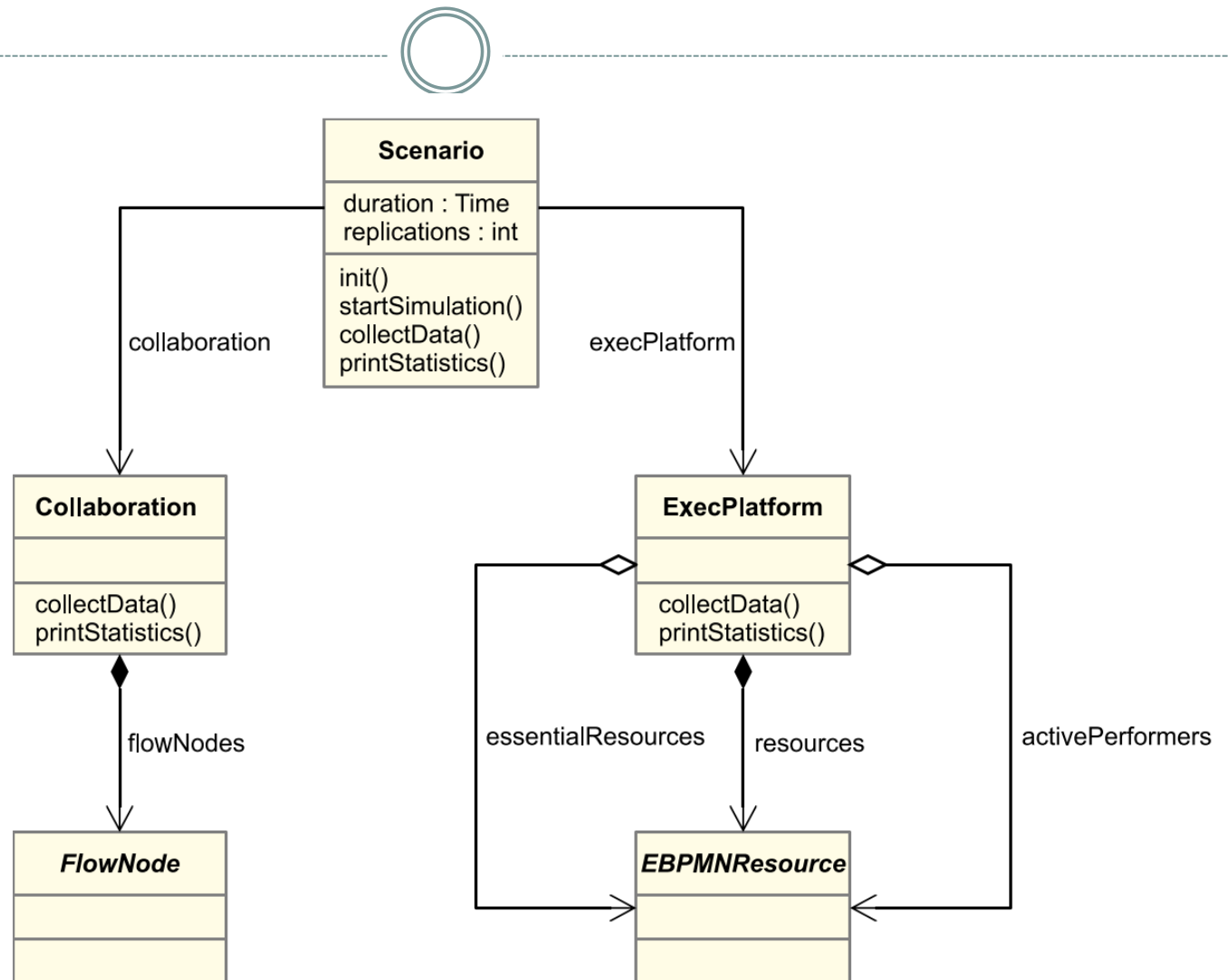
- Domain-specific simulation language designed to have an execution semantics that conforms to the BPMN 2.0 specification
- eBPMN is used to simulate:
 - A single process owned by a single participant
 - A collaboration of interacting processes
- eBPMN simulates the resource behavior and the non-functional properties defined by the PyBPMN extension metamodel

eBPMN Basic Concepts



- The eBPMN language uses the token concept to simulate the execution of a BPMN process
 - Generated by a `Start` element at simulation time, according to inter-arrival probability distribution
 - Passing through the process elements according to the sequence flow
 - Each eBPMN element handles the token according to its execution semantics
- Several events can take place during simulation (reliability-related not shown):
 - `INCOMING_TOKEN`
 - ✦ Arrival of a token at an eBPMN element
 - `INCOMING_MESSAGE`
 - ✦ Arrival of a message at a catch event or a receive task
 - `SERVICE_REQUEST`
 - ✦ Request for a service generated by an eBPMN element and sent to the destination resource
 - `SERVICE_DONE`
 - ✦ Service request completed, generated by a resource and sent back to the task

eBPMN Simulation Environment



eBPMN Simulation Environment

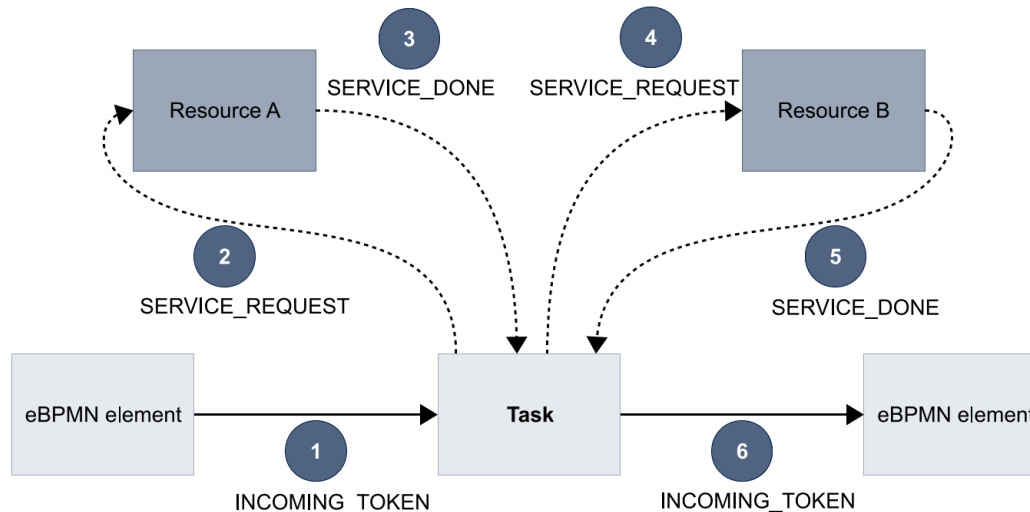


- **Simulation scenario**
 - Consisting of one or more simulation runs
- **Simulation run**
 - Consisting of one or more process instances
- **Process instance**
 - Generation of a token by a `Start` element
- **Simulation entities grouped in two sets:**
 - **Collaboration**
 - ✦ eBPMN objects corresponding to the original BPMN metamodel elements
 - **Execution platform**
 - ✦ eBPMN objects derived from PyBPMN extension metamodel

eBPMN – Service Requests



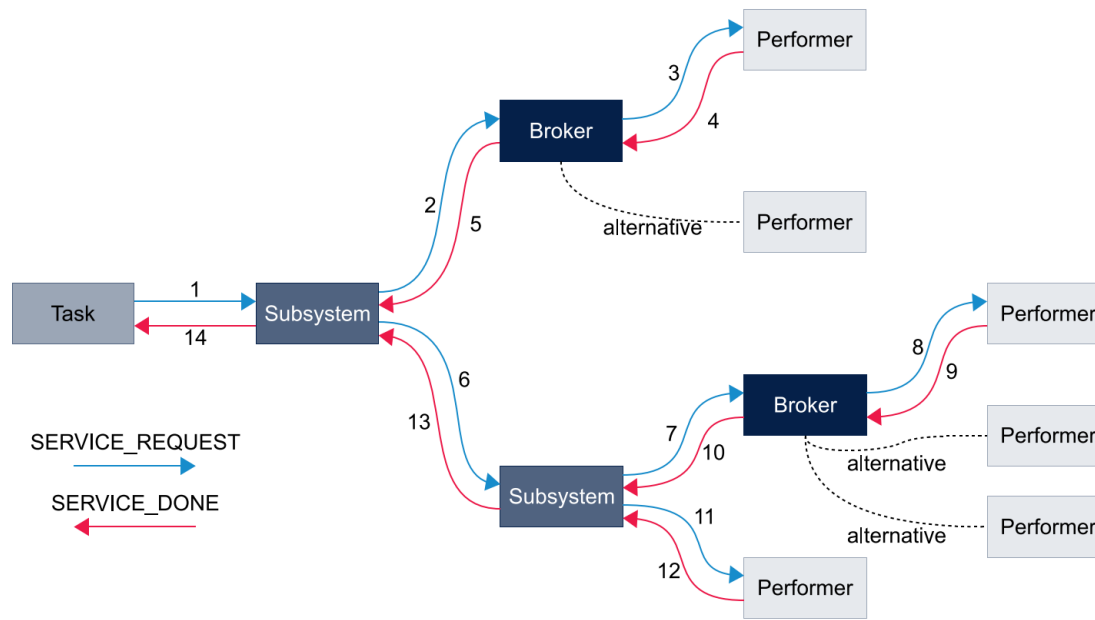
- Incoming token event for a task triggers a *service request* to the associated resource
- When the service is completed (advance in simulation time) the token passes to the next element



eBPMN – Service request delegation



- For complex resources that define a resources hierarchy, a service request delegation takes place until the request reaches a Performer



M&S Toolchain

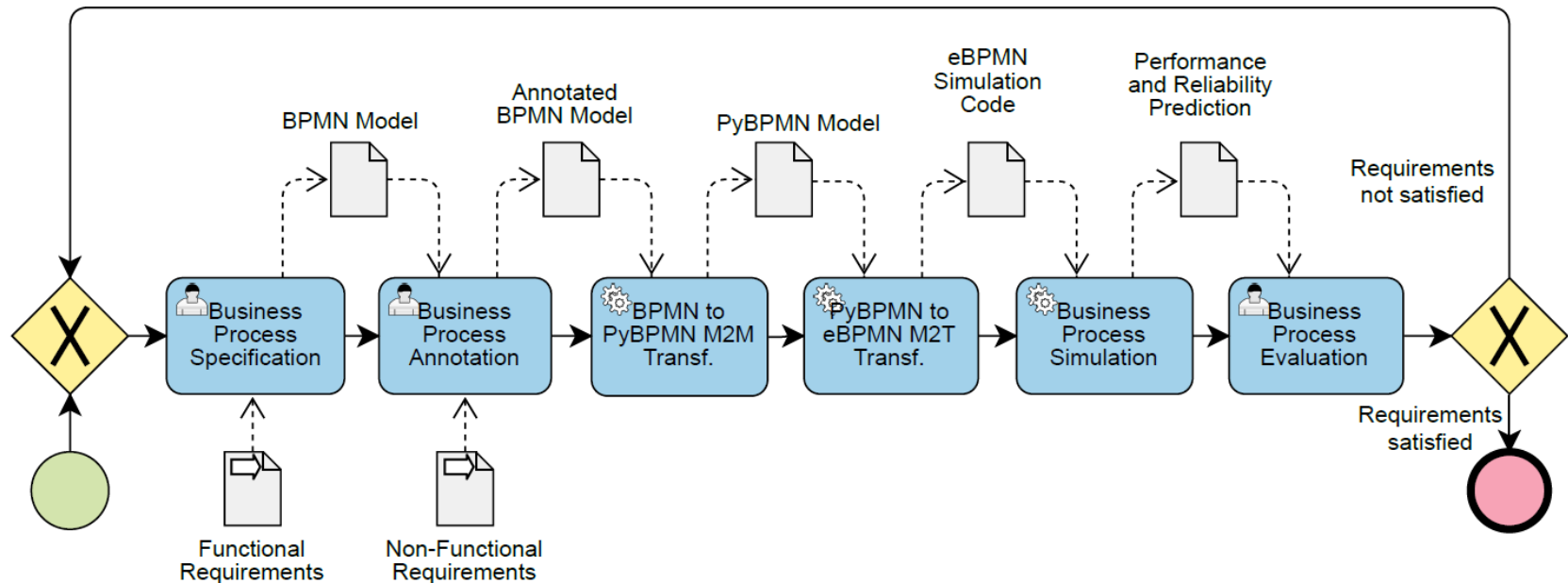


- Effective approach to bridge the gap between process model and simulation model
- Based on model-driven techniques for the automated generation of the simulation code
- Simulation code used to verify, at specification time, if the requirements set can be satisfied
- Otherwise the evaluation process can be iterated by addressing alternative scenarios in terms of process flow or resources involved in the BP

M&S Toolchain - Framework



- Model-driven business process analysis



M&S Toolchain - Core



- Model-driven method implemented using standard languages and tools within the Eclipse Modeling Framework (EMF) platform
- EMF is a framework based on Eclipse which eases the generation of programming code, tools and other applications based on structured data model
- The *Eclipse Modeling Project* is based on EMF and provides a collection of tools to enable model-driven software development capabilities within Eclipse

M&S Toolchain – BPMN2 Modeler



- Eclipse BPMN2 Modeler is an open-source, graphical tool for authoring and editing BPMN 2.0 files
- Based on the Eclipse BPMN 2.0 EMF metamodel
- Main advantages:
 - Conformance to the OMG BPMN 2.0 standard
 - Easy integration into the EMF toolchain
 - Visual modeling tool

Implemented Eclipse Plugin



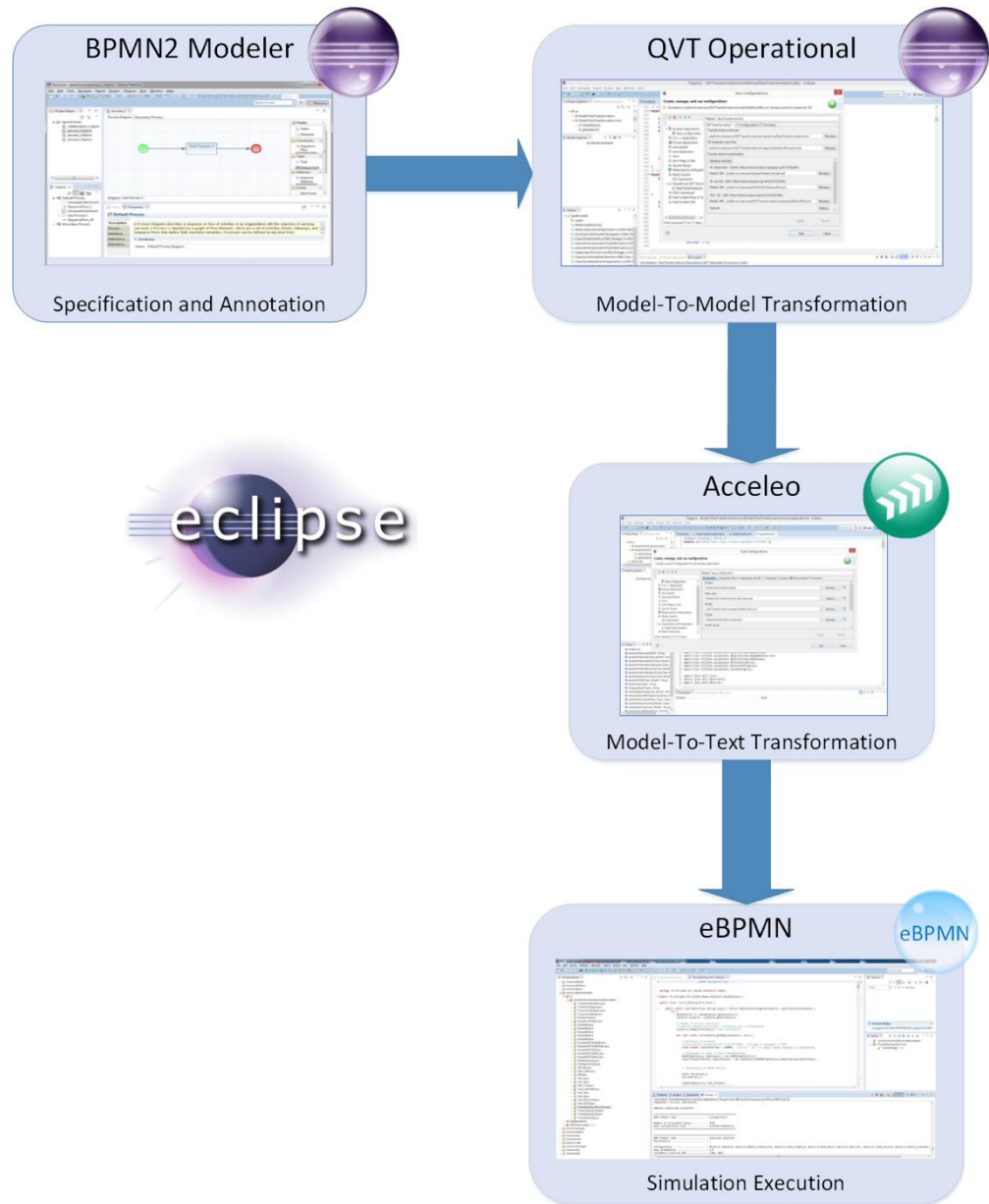
- Model-driven toolchain implemented by use of several Eclipse plugins
 - PyBPMN metamodel
 - BPMN to PyBPMN model-to-model transformation
 - PyBPMN to eBPMN model-to-text transformation
 - Eclipse PyBPMN contextual menu

M&S Toolchain



- Eclipse Modeling Project tools and implemented plugins assembled into an integrated environment
 - Used to design, simulate and evaluate BP
- The business process analyst:
 - models the business process using the BPMN2 Modeler
 - enriches the process model with textual annotation, according to the PyBPMN syntax
 - uses the contextual menu to execute transformations
 - executes the eBPMN simulation code to evaluate the business process behavior

Model-driven M&S Toolchain



Example Application: Scenario Definition



- Emergency attendance process
 - Inspired by the process used as running example in the Bizagi Process Modeler user guide
- The call center agent gathers all the information about the person affected by the emergency
- The call center agent transmits the report to a nurse
- The nurse analyzes the report and classifies the emergency according to its severity

Emergency Severity Classification



- **Green code (50%)**
 - Low severity, the patient can be easily treated
- **Yellow code (30%)**
 - Medium severity, the patient requires some special attention but can be treated at the place of emergency
- **Red code (20%)**
 - High severity, the patient must be taken to an hospital

Emergency Severity Response

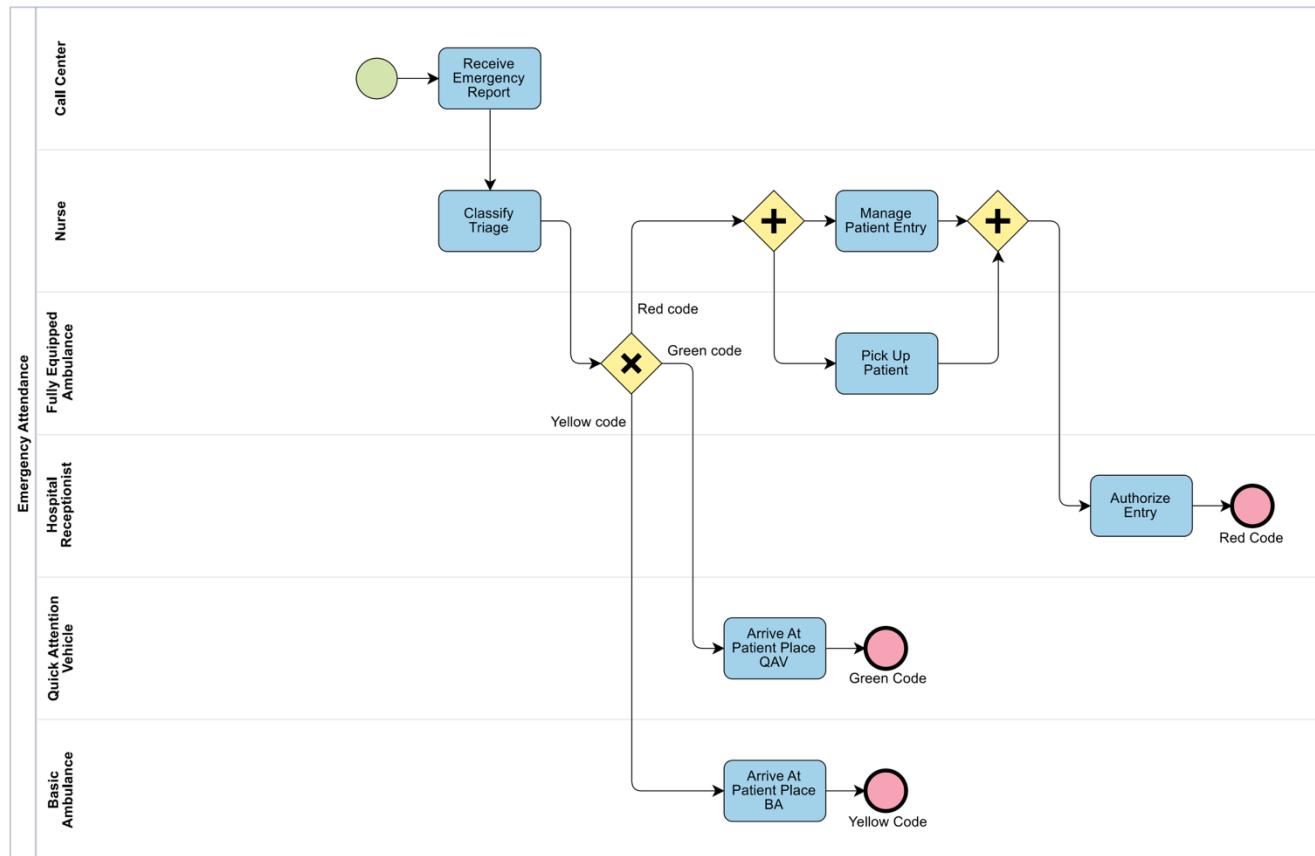


- **Green code**
 - Triage assisted by a quick attention vehicle (QAV)
- **Yellow code**
 - Triage assisted by a basic ambulance (BA)
- **Red code**
 - Triage assisted by a fully equipped ambulance
 - Fully equipped ambulance takes the patient to the nearest hospital
 - During the transfer a nurse prepares the paperwork that is used by the receptionist at the hospital to admit the patient

Example Application as a BPMN Model



- BPMN stops here



Business Process Specification



- BPMN model is enriched with non-functional properties
 - Only performance properties in this example (no reliability)
- Quantities for the available resources
- Association of activities and resources
- Amount of work for each activity

Quantities and Service Times



Resource	Quantity
Call center agent	2
Nurse	3
Fully equipped ambulance	3
Quick attention vehicle	2
Basic ambulance	2
Hospital receptionist	2

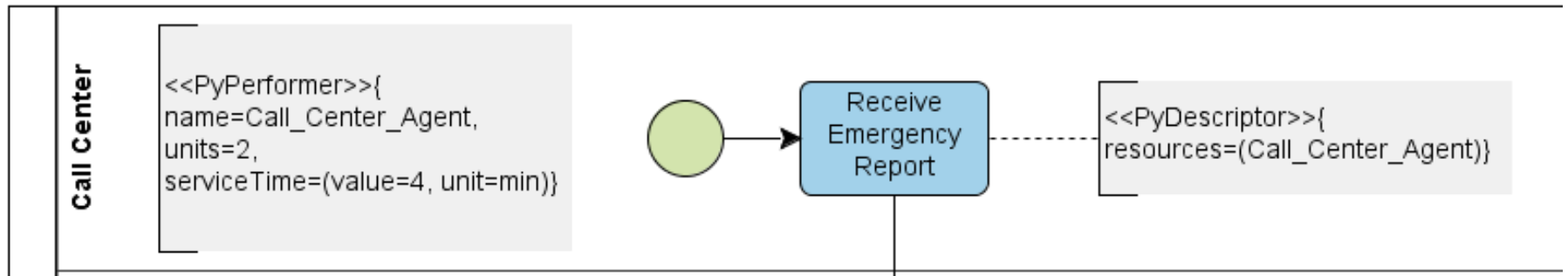
Activity	Processing time [min]	Resource
Receive emergency report	4	Call center agent
Classify triage	5	Nurse
Manage patient entry	11	Nurse
Pick up patient	20	Fully equipped ambulance
Arrive at patient place QAV	7	Quick attention vehicle
Arrive at patient place BA	10	Basic ambulance
Authorize entry	4	Hospital receptionist

Business Process Model Annotation

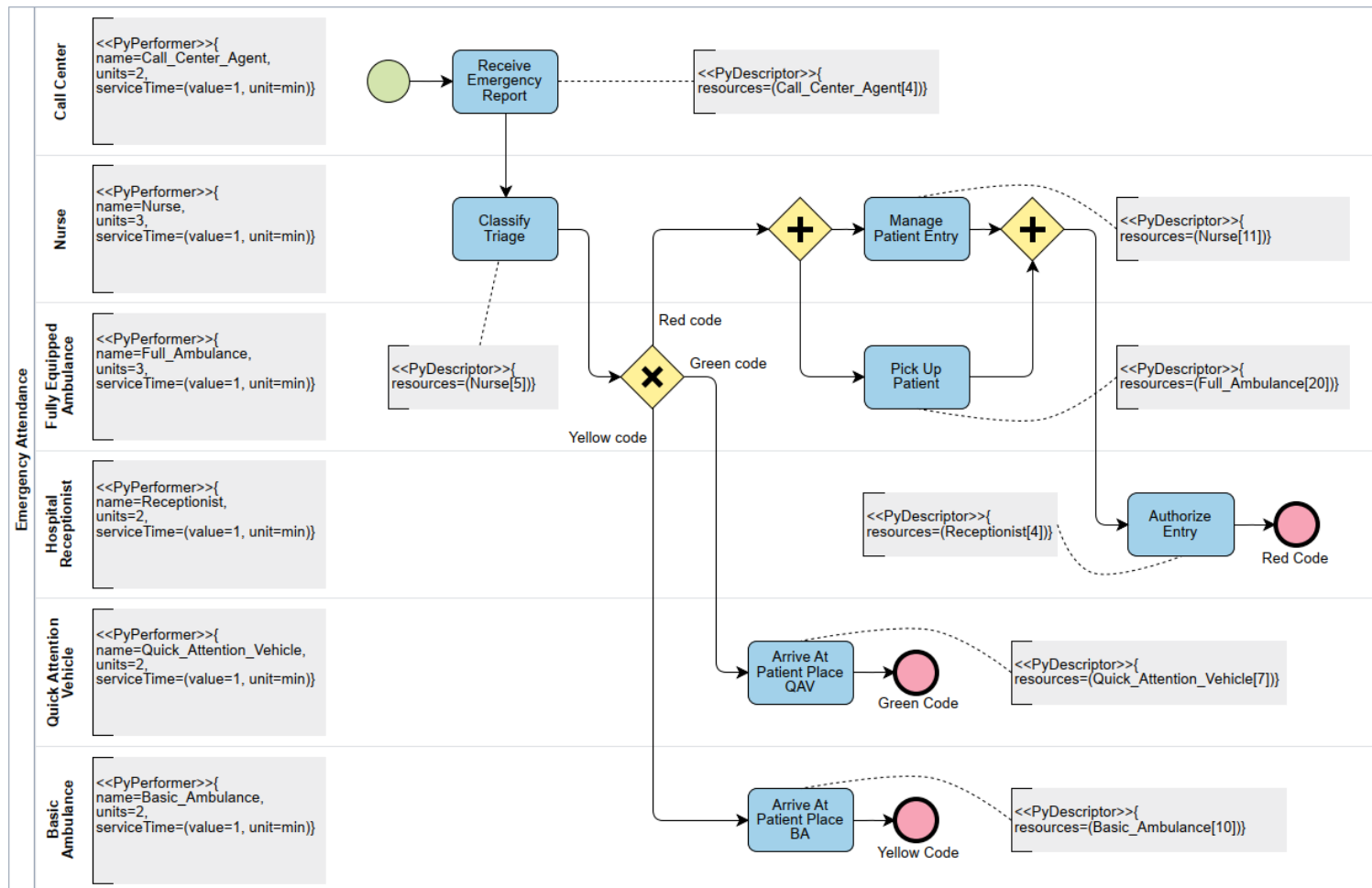


- Non-functional properties are specified in the BPMN model as text annotations that conform to the PyBPMN grammar

Resource	Quantity	Activity	Processing time [min]	Resource
Call center agent	2	Receive emergency report	4	Call center agent



Annotated BPMN Model

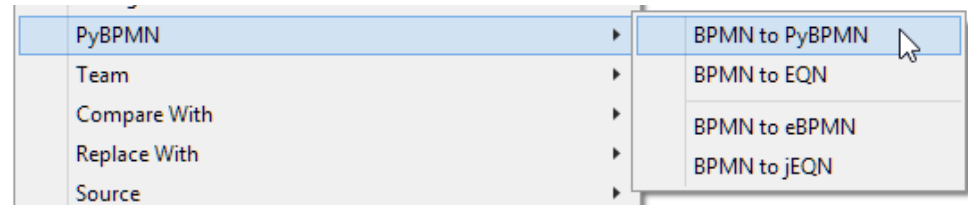
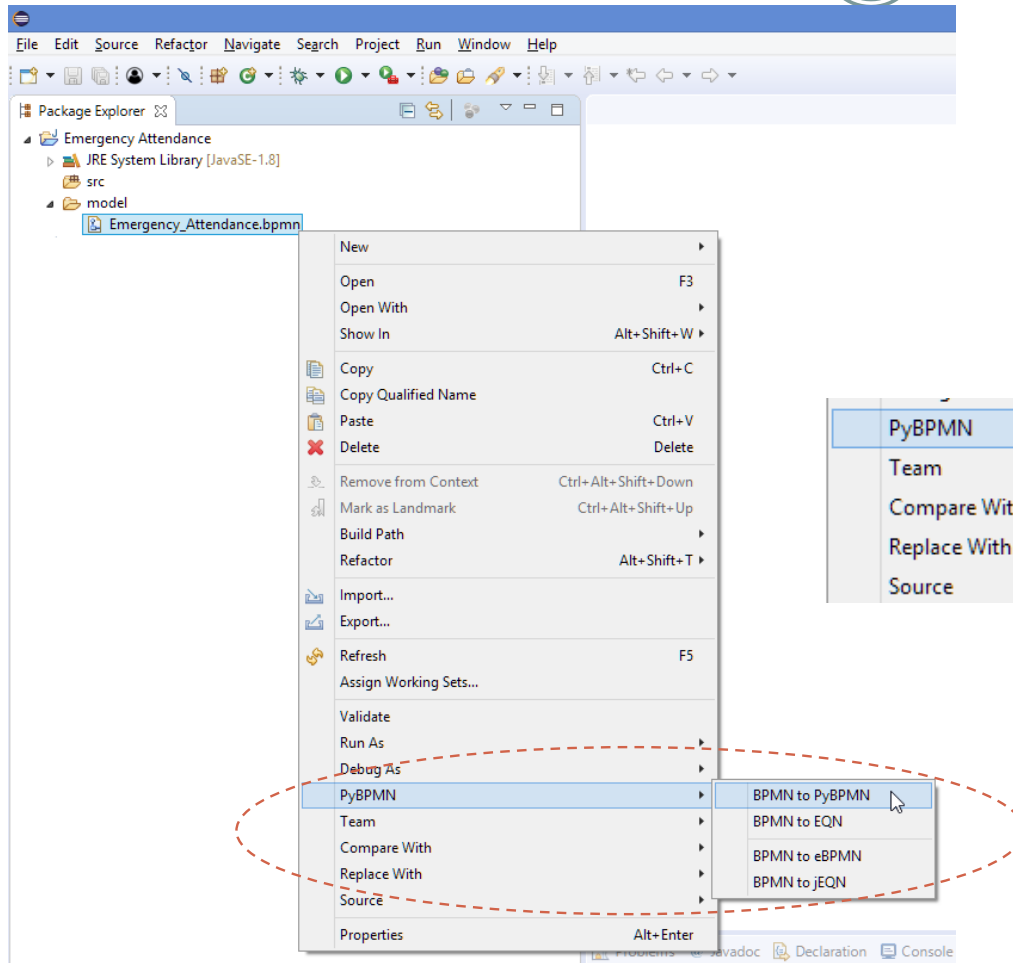


BPMN to PyBPMN

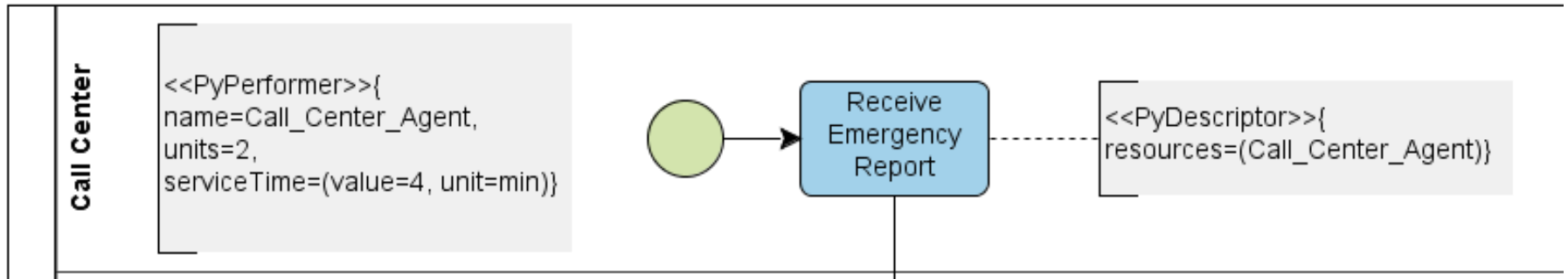


- BPMN to PyBPMN model-to-model transformation
 - Input: annotated BPMN model
 - Output: PyBPMN model
- The transformation generates a target model with PyBPMN elements that correspond to text annotations in the BPMN model

BPMN to PyBPMN Execution



PyBPMN Model



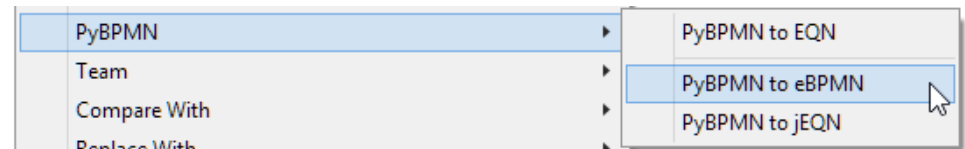
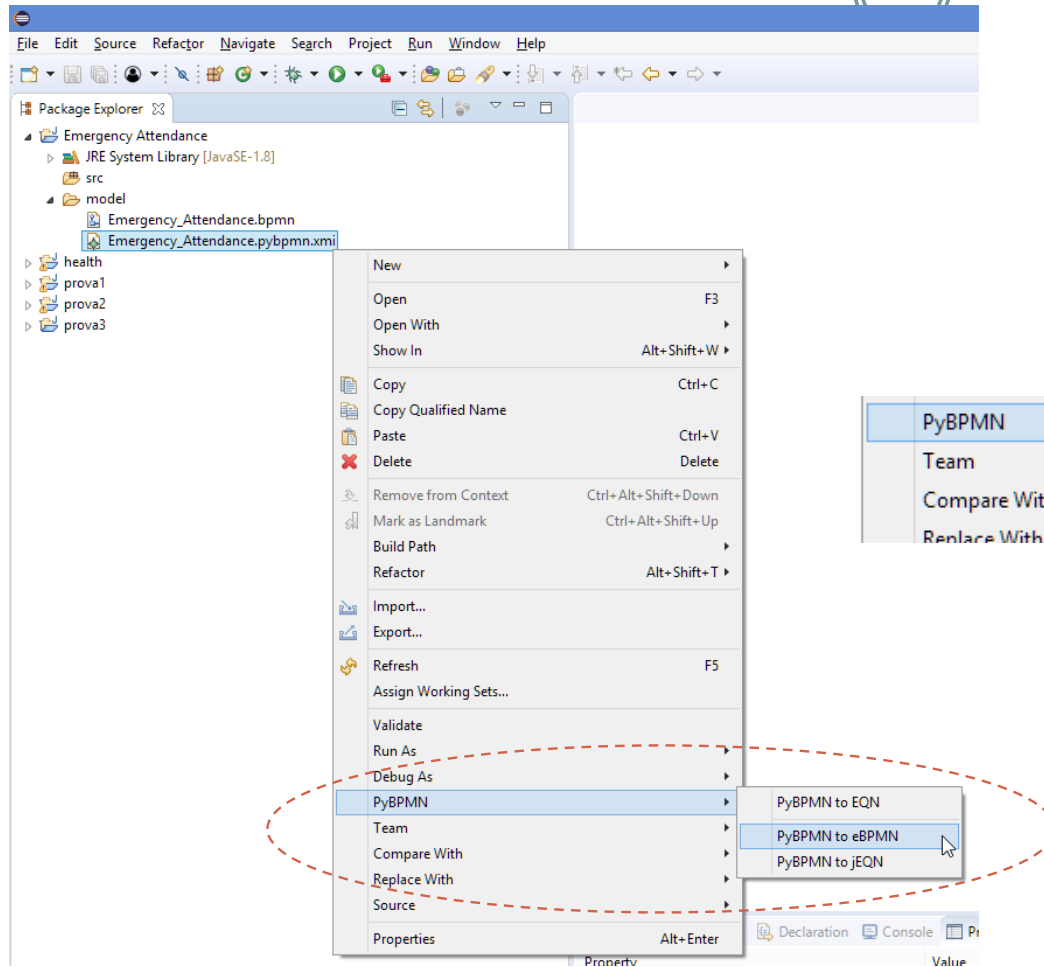
```
<elements xsi:type="pybpmn:PyPerformer" id="Call_Center_Agent" units="2">
  <performanceParams xsi:type="pybpmn:PaService" id="Call_Center_Agent_PaService">
    <serviceTime id="Call_Center_Agent_serviceTime_NFP" value="4" unit="min"/>
  </performanceParams>
</elements>
...
<elements xsi:type="pybpmn:PyDescriptor" id="Receive_Emergency_Report_Descriptor"
targetRef="Receive_Emergency_Report">
  <resources id="Req_1_from_Receive_Emergency_Report_to_Call_Center_Agent"
serviceQuantity="4" resourceRef="Call_Center_Agent"/>
</elements>
```

PyBPMN to eBPMN



- PyBPMN to eBPMN model-to-text transformation
 - Input: PyBPMN model
 - Output: eBPMN simulation code
- The transformation generates the simulation code parametrizing generic code template with PyBPMN model data

PyBPMN to eBPMN Execution



eBPMN Simulation Code



```
/**
 * Call_Center_Agent
 */
double Call_Center_Agent_service = 4; // unit is: min
double Call_Center_Agent_service_conv_factor = 1.0 * 60;

int Call_Center_Agent_workingUnits = 2;

Call_Center_Agent = new Performer(
    new EBPMNName("Call_Center_Agent"),
    Call_Center_Agent_workingUnits,
    EBPMNTime.makeFrom(
        Call_Center_Agent_service *
        Call_Center_Agent_service_conv_factor),
    // other parameters);
```

Business Process Simulation



- Execution of the auto-generated eBPMN code
- The code can also be manually modified for fine tuning
- Creation of a relevant number of process instances
 - Execution of the instances step-by-step and collection of information on process measures
- Measures used to:
 - Predict the process performance
 - Check if the initial requirements are met

Business Process Simulation Results



- *Input data*
 - *Report inter-arrival time = 3 minutes*
 - *Exponential distributions*
 - *Simulation duration = 7 days*
 - *20 replications*

```

=====
### Element name ..... : Call_Center_Agent

Final state ..... : Active (with probability 1.0000)
Failures ..... : 0
MTTF ..... : NaN day(s)
Repairs ..... : 0
MTTR ..... : NaN day(s)
Mean working units ..... : 2.00 / 2
Availability ..... : 1.0000
Resource utilization ..... : 0.6639
Processed tokens ..... : 66958
Mean of service time ..... : 3.998 min
Variance of service time ..... : 15.980
Histogram of service time .....:
| ..... : 0.00 [<0.000]
| ***** ..... : 0.39 [0.000,1.999]
| ***** ..... : 0.24 [1.999,3.998]
| ***** ..... : 0.14 [3.998,5.996]
| *** ..... : 0.09 [5.996,7.995]
| ** ..... : 0.05 [7.995,9.994]
| * ..... : 0.03 [9.994,11.993]
| * ..... : 0.02 [11.993,13.991]
| ..... : 0.01 [13.991,15.990]
| ..... : 0.02 [>15.990]

Mean of waiting time ..... : 3.186 min
Variance of waiting time ..... : 27.997
Histogram of waiting time .....:
| ..... : 0.00 [<0.000]
| ***** ..... : 0.64 [0.000,2.382]
| ***** ..... : 0.12 [2.382,4.765]
| ***** ..... : 0.08 [4.765,7.147]
| ** ..... : 0.05 [7.147,9.530]
| * ..... : 0.04 [9.530,11.912]
| * ..... : 0.02 [11.912,14.294]
| * ..... : 0.02 [14.294,16.677]
| ..... : 0.01 [16.677,19.059]
| ..... : 0.02 [>19.059]

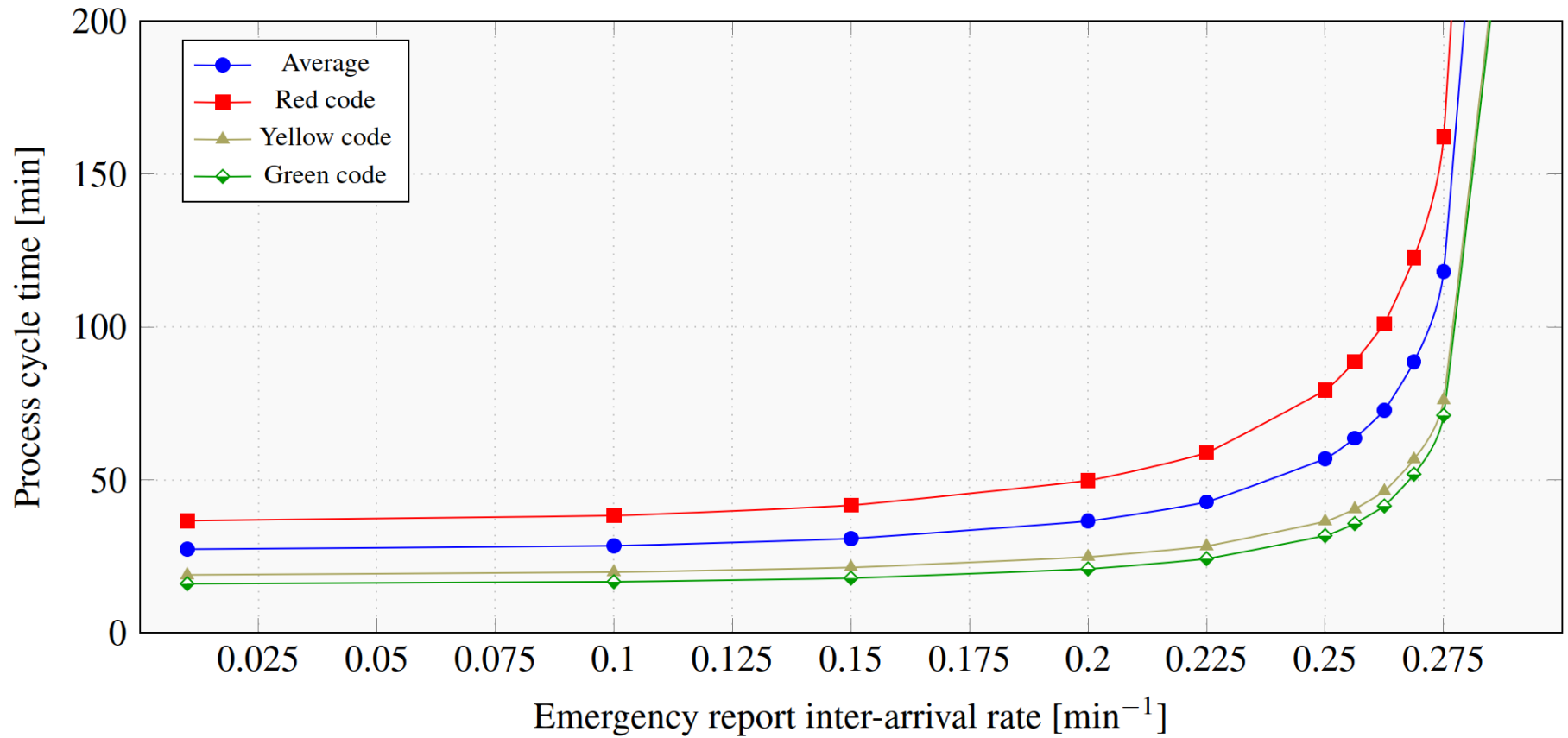
=====
    
```

Business Process Evaluation

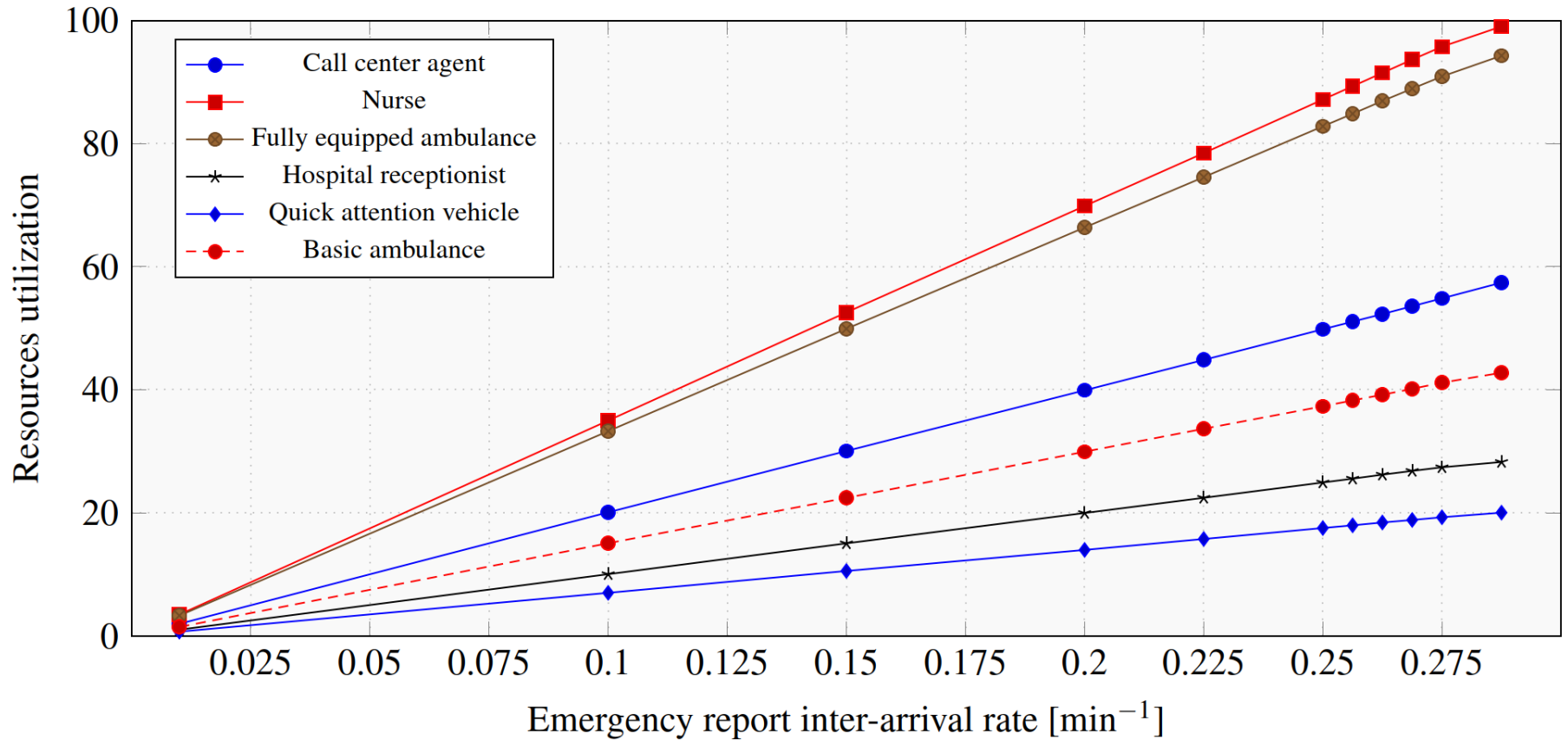


- Evaluation of process behavior under various workloads
 - Verify the BP behavior under different input conditions
- If requirements are not satisfied or the what-if analysis suggests a process improvement, the business analyst can evaluate alternative configurations by reusing the toolchain under alternative resources configuration

Cycle Times



Resource Utilization



Summary



- A largely automated approach for BPMN-based business process simulation has been illustrated
- Simulation analysis of a given business process from the annotation of the BPMN model with what is necessary to make the model executable
- Annotated BPMN automatically mapped to the simulation code ready to be executed using model transformations
- Transformations based on the **PyBPMN** metamodel, an extension of the standard BPMN to include non-functional properties, and to **eBPMN**, a domain-specific language for specifying and executing business process models
- The approach reduces the effort and the know-how typically required for using simulation-based analysis techniques

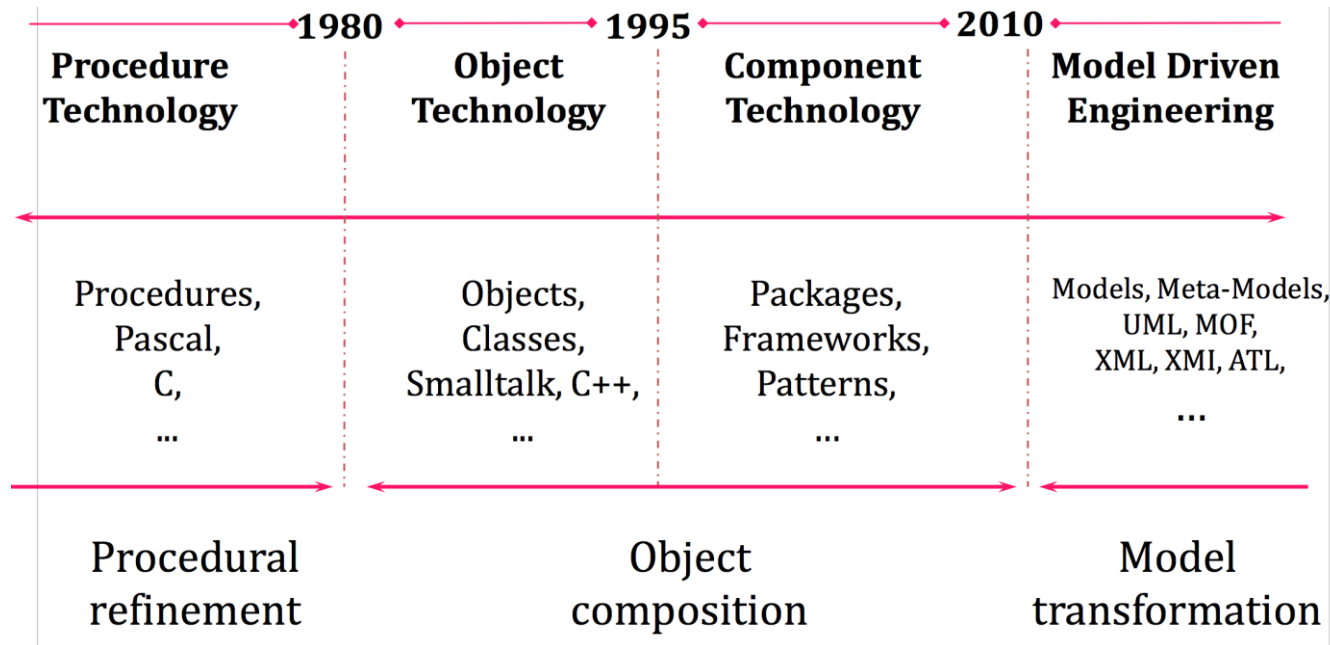


Backup slides

Model Driven Engineering (MDE)



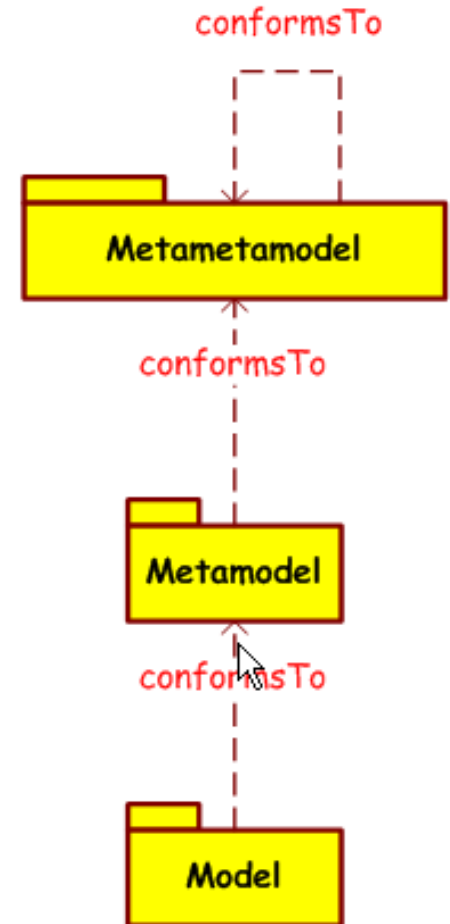
- MDE is a set of well defined practices based on tools that focus on the *productive use of models* to *increase the degree of automation* in the production, maintenance or operation of *systems and/or processes*



MDE key concepts



- MDE is based on two essential concepts:
 - **Metamodel:** a model used to define other models (i.e., the model that defines the modeling language)
 - ✦ A metamodel is itself a model
 - ✦ Each model conforms to a given metamodel
 - **Model transformation:** generation of one (or more) target model(s) from one (or more) input model(s)
 - ✦ The input and output models may be instantiated from the same metamodel (*endogenous* transformation) or from different metamodels (*exogenous* transformation)

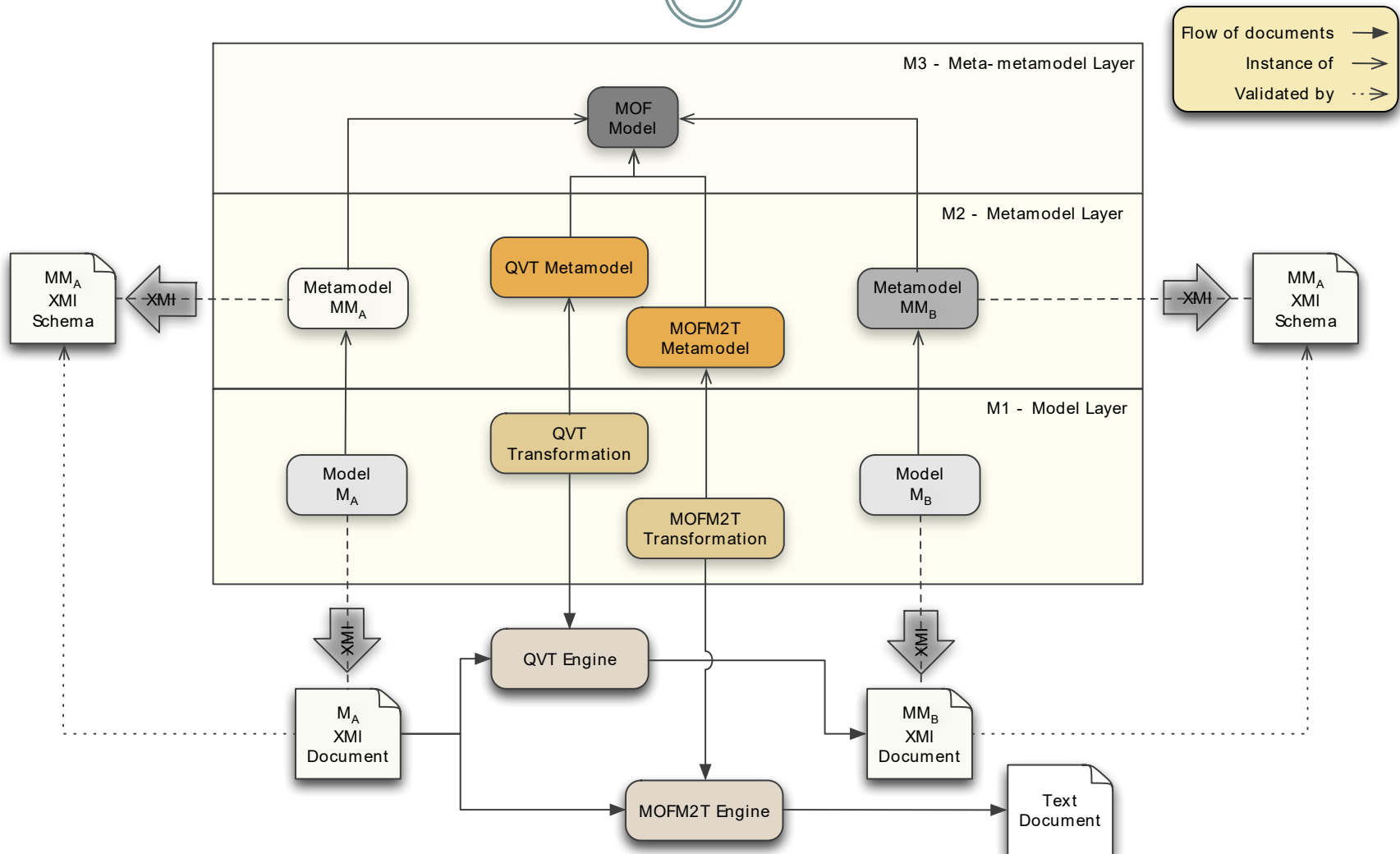


Model Driven Architecture (MDA)



- One of the most important initiatives driven by **MDE** is the OMG's **Model Driven Architecture (MDA)**
- MDA provides the following standards:
 - **Meta Object Facility (MOF)**: specification of technology neutral metamodels (i.e., models used to describe other models)
 - **XML Metadata Interchange (XMI)**: XML-based serialization of MOF models and metamodels
 - **Query/View/Transformation (QVT)**: specification of *model-to-model* transformations
 - **MOF Model To Text (MOFM2T)**: specification of *model-to-text* transformations

MDA in a nutshell



Time-stepped simulation



- Simulation time split in fixed small intervals
- Evaluation done for an interval, then advance to next interval
- Easy to implement
- Computation is wasted when no events are in a processed interval

Event Driven simulation

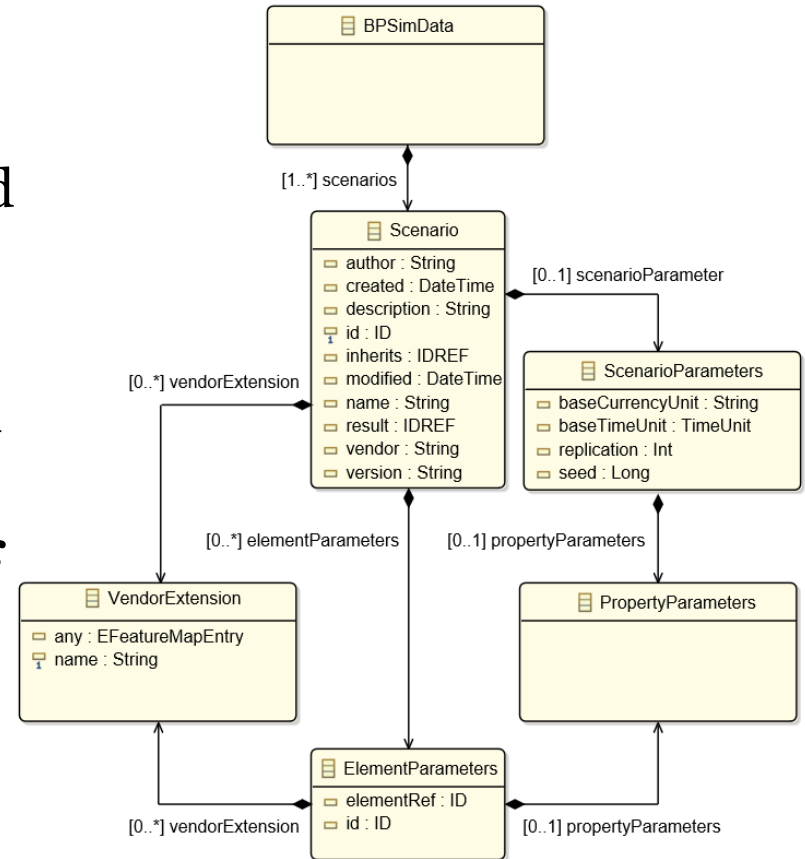


- Only (simulation) times of events are scheduled
- Evaluation done only for those times
- More complex to implement (queue of events)
- More efficient
 - No evaluation for the time interval between two consecutive events

BPSim



- Support for
 - *Pre-execution* optimization
 - ✦ “what-if” analysis using estimated values as input parameter
 - *Post-execution* optimization
 - ✦ “what-if” analysis using historical values as input parameters
- Also used to store outputs of process analysis:
 - Both predicted values and data from real executions

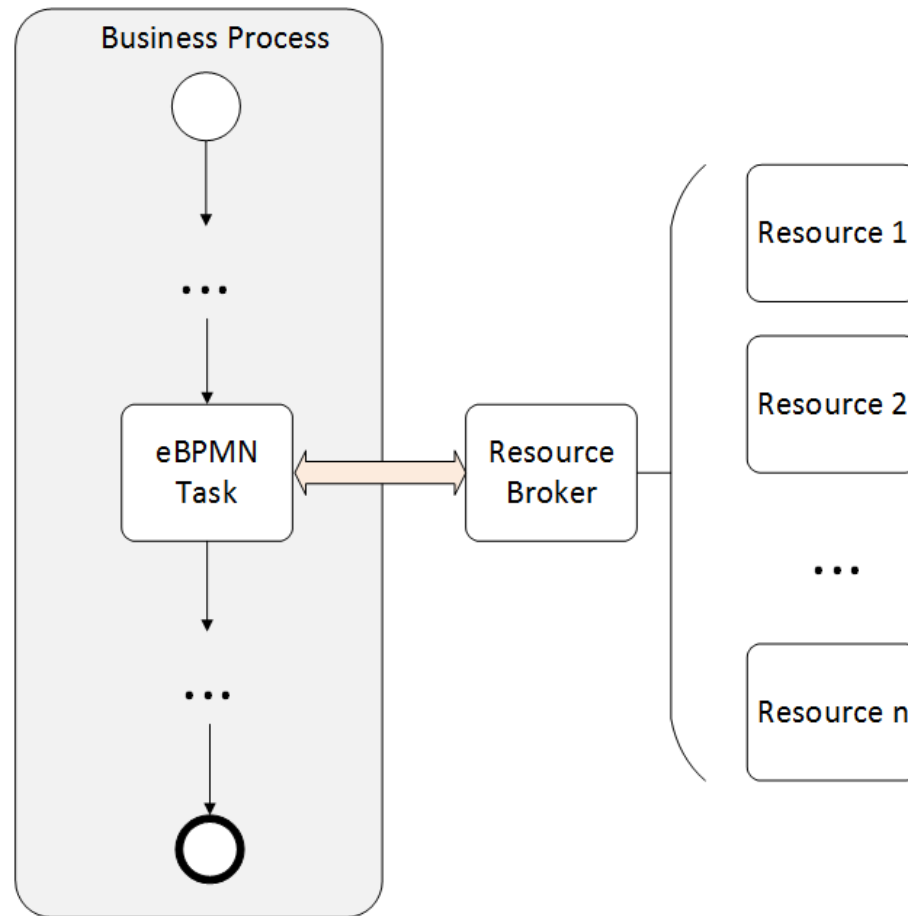


BPSim Parameters for Resources



- BPSim metamodel allows to specify some parameters that can be attached to the resources associated to a BPMN element:
 - Availability
 - ✦ Boolean, value used to specify if the resource is available or not
 - ✦ Generally varied according to a calendar
 - Quantity
 - Selection
 - ✦ Parameter used for selecting the resource to be assigned to the BPMN element
 - Role

PyBPMN Resource Broker



M&S Toolchain - Framework



- **Business Process Specification**
 - The business process is specified as a BPMN model according to functional requirements
- **Business Process Annotation**
 - The BPMN model is enriched with textual annotations specifying non-functional requirements (workload, performance, reliability and resource characterization)
- ***BPMN to PyBPMN Model-to-Model* Transformation**
 - To generate the PyBPMN model compliant with the PyBPMN metamodel from the annotated BPMN model

M&S Toolchain - Framework



- ***PyBPMN to eBPMN Model-to-Text* Transformation**
 - To generate eBPMN simulation code from the PyBPMN model
- **Business Process Simulation**
 - The eBPMN simulation code is executed to predict the behavior of the business process
- **Business Process Evaluation**
 - To verify the ability of the business process to satisfy the expected
 - In case of unsuccessful verification, simulation results are used to identify potential improvements of the business process model and evaluate relevant alternative configurations

BPMN to PyBPMN Transformation



- Model-to-model *QVT-based* operational transformation that takes a BPMN model as source and produces a PyBPMN model as target
- Transformation executes two steps:
 - Mapping of the BPMN text annotations (compliant with the PyBPMN syntax) to the corresponding PyBPMN elements
 - Creation of new references between the original BPMN elements and the just created PyBPMN elements

BPMN to PyBPMN Transformation



```
1 bpmn_in.objectsOfType(Process)->map mapAnnotations();
2 bpmn_in.objectsOfType(Definitions).missingRefs->map refineReferences();
```

```
1  /**-----
2   * Map PyAnnotations to PyBPMN elements.
3   *-----*/
4  mapping Process::mapAnnotations()
5  {
6   // Create PyData element
7   var pyData := object PyData {
8     id := 'PyData_' + self.id;
9   };
10
11   var pyConfiguration := object PyConfiguration {
12     id := 'PyConfiguration_' + self.id;
13     name := 'Simulation configuration for ' + self.id;
14   };
15
16   // Map each annotation which is a valid PyAnnotation
17   var proc := pybpmn_out.objectsOfType(Process)->selectOne(i | i.id = self.id);
18   var annotationsList :=
19     proc.artifacts[TextAnnotation]->select(i | i.isPyAnnotation())->map
20     Annotation_to_PyElements();
```

PyBPMN to eBPMN Transformation



- *Model-to-text transformation* that takes a PyBPMN model as input and generates the corresponding eBPMN simulation code
- *Template-based approach* that uses placeholders to specify which data, as metamodel entities, have to be extracted from the model and put in code

PyBPMN to eBPMN Transformation

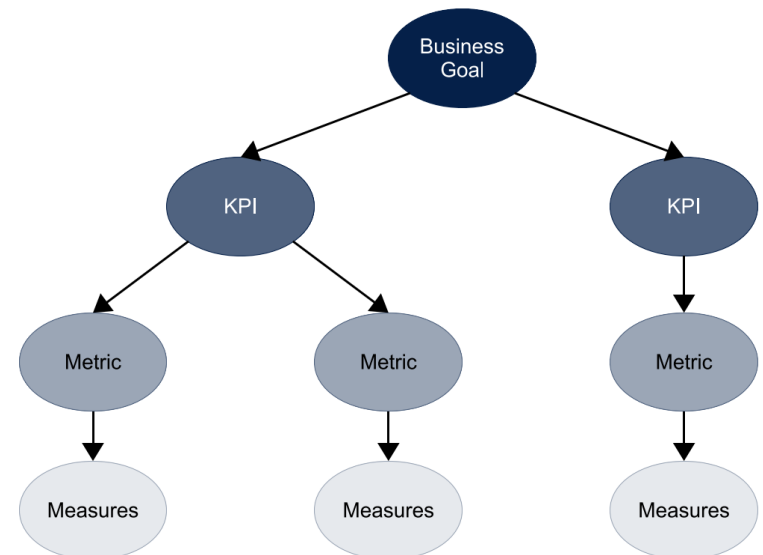


```
9  [comment PyPerformer implementation /]
10  [template public generatePerformer(aResource : PyPerformer)]
11    [header(aResource.id) /]
12
13    [let perfParam : PaQualification = aResource.performanceParams]
14      [let service_value : String = perfParam.oclAsType(PaService).serviceTime.value]
15      [let service_unit : TimeUnitKind = perfParam.oclAsType(PaService).serviceTime.
16        unit]
17      double [aResource.id /]_service = [service_value /]; // unit is: [
18        service_unit.toString() /]
19      [timeUnitConvFactor(aResource.id + '_service', service_unit) /]
20    [/let]
21  [/let]
22  [/let]
```

Example Application



- Application of the proposed model-driven method for business process evaluation
- Evaluate what and how?
 - The achievement of a business goal, comparing KPI with established thresholds



Methodology for the Example Application



- Business goal -> target that the organization aims to reach
 - To effectively check if such target is reached, the business goal has to be measurable
- Key Performance Indicator (KPI) -> captures how effectively a business process is performing
 - Provides a numerical value that can be compared to the threshold for the achievement of the target
- Business metric -> measurement used to track and assess the status of a specific business process
 - Summarizes the measures gathered on the business process